TeCCo: A Terminal-Cloud Cross-Domain Collaborative Framework for Remote Sensing Image Classification

Ting Zhang, Peirui Cheng, Liangjin Zhao, Zhirui Wang, *Member, IEEE*, Lingyu Kong, *Student Member, IEEE*, Chenglong Liu, *Graduate Student Member, IEEE*, Guangluan Xu, and Xian Sun, *Senior Member, IEEE*

***Abstract*— The terminal-cloud collaborative framework boosts precision and efficiency by integrating cloud computing power with low-latency terminal responsiveness, offering a suitable solution for the growing demands of multiplatform remote sensing (RS) image interpretation. However, the significant differences in data distribution across various RS platforms present a great challenge in balancing the cloud's centralized processing capabilities with the local interpretation abilities of different terminals. To address this challenge, we propose a terminal-cloud cross-domain collaborative (TeCCo) framework that inherits the efficiency advantages of multiple platforms while ensuring high-accuracy interpretation of diverse data distributions from different terminals. First, the dual classifier co-learning (DCCL) module is designed to enhance cloud robustness. By combining a multilayer perceptron for instance-level classification and a graph convolutional network (GCN) for feature-level aggregation, it achieves mutual supervision and improves feature alignment across different data distributions. Second, the hypernetwork personalization (HNP) module is introduced to generate personalized classifier parameters for each terminal with little fine-tuning cost, allowing terminals to maintain their uniqueness while benefiting from the generalization advantages of collaborative training. Finally, a data-assisted progressive inference mechanism is proposed to enhance accuracy by jointly clustering the features transmitted from terminals and the features of supervised data in the cloud. Extensive experiments demonstrate that TeCCo effectively addresses data distribution challenges, enhancing both the generalization of the cloud model and the personalization of terminal models, achieving state-of-the-art (SOTA) performance in cross-domain and multiplatform RS image classification.**

Received 30 July 2024; revised 20 November 2024; accepted 7 December 2024. Date of publication 11 December 2024; date of current version 19 December 2024. This work was supported in part by the National Nature Science Foundation of China under Grant 62331027 and in part by the Strategic Priority Research Program of the Chinese Academy of Sciences under Grant XDA0360300. *(Corresponding authors: Zhirui Wang; Liangjin Zhao.)*

Ting Zhang, Lingyu Kong, Chenglong Liu, and Xian Sun are with the Key Laboratory of Target Cognition and Application Technology (TCAT), Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing 100190, China, and also with the School of Electronic, Electrical and Communication Engineering, University of Chinese Academy of Sciences, Beijing 100190, China (e-mail: zhangting20@mails.ucas.ac.cn; konglingyu20@mails.ucas.ac.cn; liuchenglong20@mails.ucas.ac.cn; sunxian@aircas.ac.cn).

Peirui Cheng, Liangjin Zhao, Zhirui Wang, and Guangluan Xu are with the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing 100094, China, and also with the Key Laboratory of Target Cognition and Application Technology (TCAT), Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing 100190, China (e-mail: chengpr@aircas.ac.cn; zhaolj004896@aircas.ac.cn; zhirui1990@126.com; gluanxu@mail.ie.ac.cn).

Digital Object Identifier 10.1109/TGRS.2024.3515464

***Index Terms*— Domain adaptation, multiplatform image classification, remote sensing (RS), terminal-cloud collaboration.**

I. Introduction

As REMOTE sensing (RS) technology advances, Earth observation systems based on single-platform networks have achieved progress in various real-world applications, such as environmental monitoring [1], [2], objective detection [3], [4], and urban planning [5], [6]. However, single-platform networks often face computational and storage limitations, which restrict their processing efficiency and observational capabilities. To overcome these challenges, Earth observation systems are evolving toward multiplatform networks to meet the growing demands for high-resolution and real-time applications [7], [8], [9]. In particular, the terminal-cloud collaboration framework [10], [11], [12], [13] has emerged as an effective solution for enhancing the capabilities of multiplatform networks.

In the terminal-cloud collaborative framework, RS devices serve as terminals equipped with lightweight models for real-time image interpretation. When these terminals encounter complex or ambiguous images that are difficult to classify accurately, they can transmit them to the central cloud, such as a resource-rich RS device or a ground processing center. The cloud device, utilizing its relatively abundant resources, executes more complex inference tasks to ensure high-precision classification. This collaborative scheme leverages the strengths of both terminal and cloud, overcoming the limitations of individual devices and enhancing overall system performance. Several studies [14], [15], [16] have validated the effectiveness of this framework by partitioning neural network layers across different devices, thereby maximizing the utilization of each device's computational resources. Some researchers [17], [18] have integrated dynamic inference strategies into the terminal-cloud framework, enabling faster and more robust distributed inference. Furthermore, Zhang et al. [10] proposed a terminal-cloud framework specifically for RS image interpretation, which utilizes multiple terminal models to significantly improve processing efficiency compared to traditional single models.

However, the existing RS terminal-cloud collaborative framework with homogeneous terminals still faces a significant

1558-0644 © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

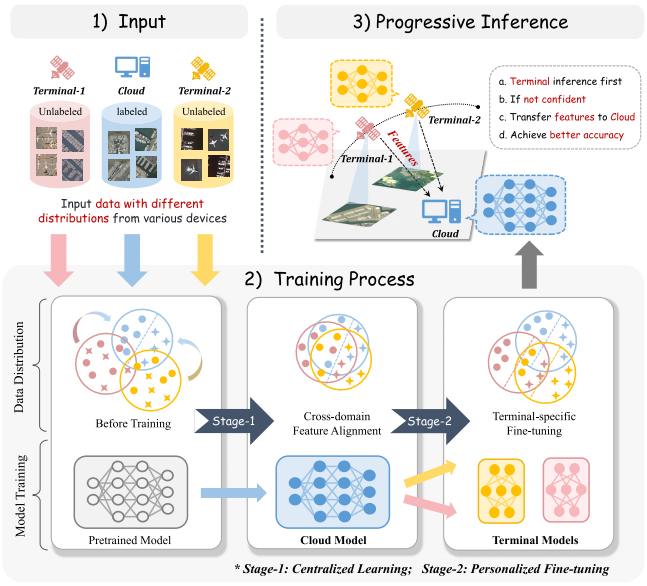


Fig. 1. Two-stage training and inference process of the proposed TeCCo.

problem, as it lacks the capability to process RS images from diverse distributions. Specifically, in practical applications, the data collected by different RS platforms often exhibit significant differences in domain distribution [19], [20]. These differences may arise from variations in environmental factors such as lighting conditions, cloud cover, and surface reflectivity, as well as the diversity of observation conditions, spatiotemporal resolution, and sensor types [21], [22]. Furthermore, the distinct characteristics of different RS platforms can also lead to significant variations in the distribution of collected data. Training deep neural networks on the cloud relies on high-level correlations within specific data to identify and extract patterns. When these correlations are weak or inconsistent across data distributions from different terminals, the model's accuracy can be significantly compromised [23], [24]. Therefore, an appropriate cloud model should possess strong generalization and robustness, allowing it to better handle diverse data distributions during both the joint training and inference phases. In addition, the heterogeneity in data distributions often results in a significant decline in prediction accuracy when a trained terminal model is directly applied to other terminals [25], [26]. This highlights the necessity for terminal models to better adapt to local data characteristics while still benefiting from global joint training.

Considering the aforementioned problems, we propose a terminal-cloud cross-domain collaborative (TeCCo) framework for RS image classification. As shown in Fig. 1, TeCCo adopts a two-stage learning approach, involving centralized learning and personalized fine-tuning. During the training process, TeCCo utilizes the cross-domain learning scheme to extract data features from diverse domains in a unified representation space. First, a dual classifier co-learning (DCCL) module is designed to enhance the prediction accuracy across multiple domains. The DCCL module utilizes a multilayer perception (MLP) classifier and a graph convolutional network (GCN) for instance-level classification and semantic-level feature clustering, respectively. By exchanging pseudo-labels to enhance the robustness of feature representation, these two classifiers achieve effective knowledge transfer and sharing through feature alignment and feedback supervision, significantly improving the cloud model's consistency and generalization ability in interpreting data from different terminals. Second, we designed a hypernetwork personalization (HNP) module, which generates specific classifier parameters for each terminal to achieve personalized adjustment. This allows each terminal to maintain its uniqueness while benefiting from collaborative training. The HNP module effectively maps the large parameter solution spaces of individual terminal classifiers into a smaller search space, thereby reducing the complexity of joint training. Finally, a data-assisted progressive inference mechanism is improved, where the features returned from the terminals and the features from the supervised source domain are jointly clustered in a more robust classifier, thereby achieving a higher accuracy guarantee on the cloud.

The main contributions of this article are summarized below.

1) This article proposes a novel TeCCo for image classification in RS multiplatform. TeCCo enhances the robustness of the cloud model and the personalization of terminal models, achieving accurate and efficient collaborative interpretation.
2) A DCCL module including GCN and MLP classifiers is introduced, improving cloud model feature representation alignment for RS images from different terminals. The mutual supervision of instance-level relationships and semantic-level features between these two classifiers enhances interpretation consistency and accuracy.
3) An HNP module is proposed that dynamically generates customized classifier parameters for each terminal, allowing the terminal models to achieve personalization while also benefiting from joint training. This effectively addresses domain-specific classification challenges within a unified model architecture.
4) A data-assisted progressive inference mechanism is designed, combining labeled data from the source domain as supplementary guidance to achieve more robust feature clustering in the GCN classifier, further enhancing the classification accuracy of the cloud model for uncertain data from various terminals.

We organize the structure of this article as follows. Section II reviews related works. Section III describes the proposed TeCCo method. Section IV presents the experimental results and provides a comparative analysis. Finally, Section V concludes the article.

II. Related Work

A. Terminal-Cloud Collaborative Models

Terminal-cloud collaboration is a promising approach to enhance processing capabilities by leveraging the strengths of both terminal and cloud resources. This collaborative framework facilitates the distribution of computational tasks, enabling real-time data processing at terminals while relying

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

on the cloud for more complex computations. Some studies [27], [28], [29], [30], [31] have proposed partitioning deep neural networks between edge devices and the cloud to optimize computational efficiency and latency. Zhao et al. [32] introduced EdgeML for real-time edge device deep learning, which optimizes performance through workload offloading and dynamic neural architecture adjustments. Mehta and Shorey [33] presented DeepSplit, a dynamic algorithm for splitting convolutional neural networks (CNNs) between edge and cloud, optimizing bandwidth consumption while maintaining prediction accuracy by selecting optimal split layers based on task load, input dimensions, and bandwidth constraints. Zhang et al. [34] breaks down the overall video analysis task into multiple subtasks, which are then sequentially assigned to different devices for execution. In addition, Zhao et al. [35] enhance efficiency by partitioning the input data into several segments and performing inference in parallel across different models.

These above methods ensure distributed accuracy and efficiency through hierarchical decomposition, task decomposition, data partitioning, or computation offloading optimization. However, they rarely consider terminal-cloud joint training tailored to specific tasks at the model level. In the field of RS, DCNNet [10] proposed a hierarchical neural network suitable for RS image classification, achieving accurate and efficient image interpretation across terminal-cloud devices. However, the parameters of each terminal are homogeneous, making it challenging to adapt to application scenarios with different data distributions across terminals. This article proposes a method for cross-domain image classification tasks in RS multiplatform scenarios, focusing on optimizing the structure and training strategies of terminal-cloud collaborative models.

B. Cross-Domain Generalization for RS Classification

The terminal-cloud framework performs well in multiplatform applications but still encounters significant challenges in cross-domain generalization. This difficulty arises due to substantial differences in data distribution across various RS platforms, leading to increased complexity in distributed model learning and reduced processing accuracy.

To achieve better generalization in cross-domain classification tasks, unsupervised domain adaptation methods have been proposed, demonstrating marked performance improvements [36], [37], [38], [39], [40], [41]. From the perspective of learning objectives, several researches can be categorized into three main types: instance-based [42], [43], [44], classifier-based [45], [46], and feature-based methods [47], [48], [49], [50]. These methods primarily focus on handcrafted features. From the perspective of learning schemes, two main categories are discrepancy-based methods [51], [52], [53] and adversarial-based methods [40], [54], [55], [56], [57], [58]. Adversarial-based methods, in particular, utilize the interplay between a generator and a discriminator to learn domain-invariant features, making them well-suited for cross-domain scenario. Huang et al. [59] introduced an adversarial domain adaptation framework with calibrated prototypes and dynamic instance convolution for cross-domain hyperspectral image classification, enhancing class separability and leveraging robust domain-invariant feature extraction. Biswas and Tešić [20] proposed an unsupervised domain adaptation method using debiased contrastive learning and support-set guided pseudolabeling. In addition, Zheng et al. [21] proposed a two-stage adaptation network for RS image classification in cross-domain learning scenarios, combining adversarial learning and self-supervised learning to handle multiple target domains.

The above methods primarily focus on extracting invariant features by reducing distribution discrepancies among different domains. However, in a distributed scenario, each terminal concentrates solely on its performance accuracy on local data, without considering its classification effectiveness across all domains. The challenge with existing global alignment cloud-terminal frameworks is that they may not capture the domain-specific features most relevant for each terminal. Therefore, there is a need for additional personalized optimizations to identify the optimal feature-separable spaces tailored to different domains in each terminal.

C. Hypernetwork

Hypernetwork, as described in various studies [60], [61], [62], [63], [64], is a type of neural network designed to produce parameters for another neural network. Initially introduced by Ha et al. [60], hypernetwork demonstrates its potential in model compression by reducing the number of parameters required for training. Recently, interest and research in hypernetworks have steadily grown. For instance, Chang et al. [63] explored parameter initialization techniques for hypernetwork. Furthermore, hypernetworks have been applied to a wide range of tasks, including continual learning [62], graph processing [65], meta-learning [66], and federated learning [64]. Recent studies have focused on generating various network parameters from different data inputs. HyperStyle [61] employs hypernetworks to generate distinct decoder parameters for different images, thereby improving the quality of image reconstruction. In our study, we utilize hypernetworks to generate personalized parameters for individual terminal classifiers within the terminal-cloud collaborative framework.

III. Proposed Method

A. Overview

In this section, we introduce the cross-domain terminal-cloud collaborative framework named TeCCo, with the architecture overview illustrated in Fig. 2. TeCCo comprises a central cloud model and multiple heterogeneous lightweight terminal models. During the training phase, TeCCo addresses the challenge of data distribution differences across various terminals in practical RS applications. Within the unified terminal-cloud collaborative framework, the training pipeline is designed to enhance the generalization capabilities of the cloud model, while each terminal model concentrates on improving the accuracy of local data. The overall training process is divided into two stages: centralized learning and personalized fine-tuning. During the inference phase,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

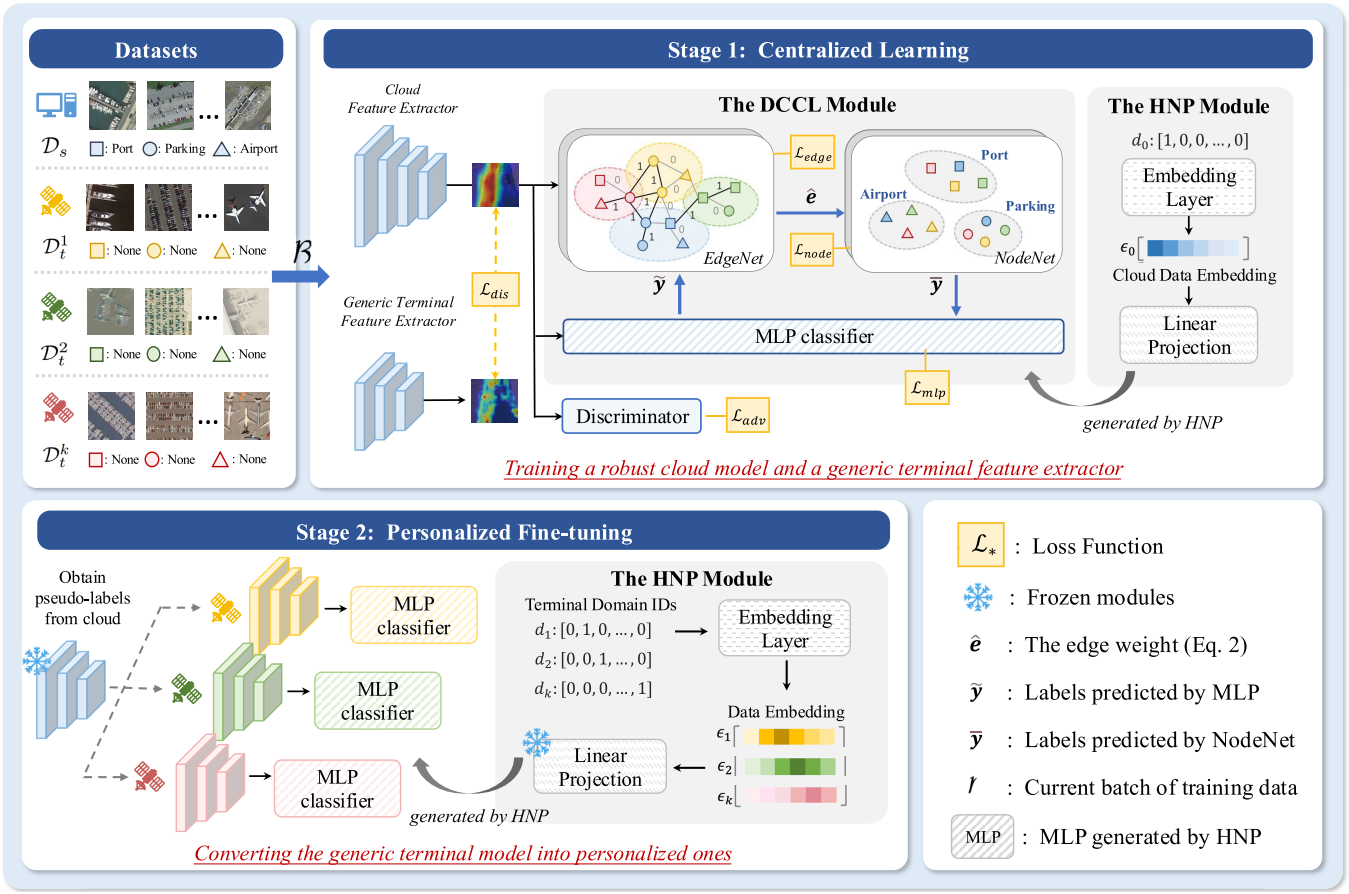


Fig. 2. Overview of TeCCo. During the first stage of centralized learning, a cloud model and a lightweight terminal feature extractor are jointly trained, with feature alignment achieved through distillation loss. The MLP classifier parameters on the cloud are generated by a trainable HNP module. The MLP and GCN classifiers mutually supervise each other at both the instance and semantic levels within the DCCL module. In the second stage of personalized fine-tuning, the cloud model uses the GCN node classifier to generate pseudo-labels for data on the terminal. Terminal models are initialized with the generic lightweight feature extractor trained in the first stage. The HNP module trains an embedding layer based on each terminal's domain ID, generating personalized classifier parameters for each terminal.

we implement a novel progressive inference mechanism that leverages the varying computational power of terminal and cloud models. A brief introduction to each component is as follows.

1) Centralized Learning: In the centralized learning stage, the focus is on training a robust cloud model and a generic terminal feature extractor. The DCCL module (detailed in Section III-B) is employed to address the inconsistent data distribution among various terminals by integrating a GCN classifier with an MLP classifier. A discriminator is incorporated to assist the cloud model in learning feature representations that are challenging to distinguish between data from different terminals. In addition, a distillation loss further supervises the alignment of features between the cloud and terminal models.

2) Personalized Fine-Tuning: After centralized learning, the personalized fine-tuning stage aims to adapt the terminal models to specific local data distributions. The HNP module (detailed in Section III-C) generates personalized classifier parameters for each terminal, allowing the terminal models to update their parameters using high-confidence pseudo-labels from the cloud model, thereby achieving diversity and flexibility among terminal models with few training parameters.

3) Progressive Inference: When TeCCo is deployed, terminals process local RS images in parallel for real-time interpretation under a progressive inference mechanism. Specifically, terminals transmit data with low prediction confidence to the cloud for more accurate predictions. Since the terminal's feature network is distilled from the cloud's feature network, only the features need to be transmitted, significantly reducing transmission resource requirements. In addition, by combining data features from the cloud device, we can refine predictions for the GCN classifier. In this way, we design the novel data-assisted progressive inference mechanism (detailed in Section III-D), leading to higher overall performance.

B. Dual-Classifier Co-Learning Module

In the practical implementation of RS, inconsistent data distributions among various terminals often lead to performance gaps in the terminal-cloud collaborative framework. To address this issue, the DCCL module is introduced. It employs adversarial cross-domain learning to facilitate knowledge transfer from cloud models to terminal models and uses a cooperative framework that integrates a GCN classifier with an MLP classifier to enhance cloud model training. The GCN classifier includes EdgeNet for instance-level relation learning and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

NodeNet for feature aggregation. This setup allows the GCN and MLP classifiers to interact and mutually supervise, leading to stronger feature representation through the joint learning of relationships and individual features.

Specifically, let $\mathcal{D}_s = \{(x_i^s, y_i^s)\}_{i=1}^{N_s}$ denote the labeled dataset on the cloud as the source domain, where x_i^s is the ith input, y_i^s is its corresponding label, and N_s is the number of samples in the cloud dataset. Similarly, let $\mathcal{D}_t^k = \{x_i^{t_k}\}_{i=1}^{N_{t_k}}$ represent the target domain dataset from the kth terminal, where $x_i^{t_k}$ is the ith input in the kth target domain and N_{t_k} is the number of samples in that target domain. Each input image x (be it x_i^s or $x_i^{t_k}$) is transformed through a feature extractor F_{feat} into a high-dimensional feature vector

$$f_i = F_{\text{feat}}(x_i; \Theta_{\text{feat}}) \tag{1}$$

where f_i represents the feature vector corresponding to the input x_i, and Θ_{feat} denotes the parameters of the feature network extractor.

Each input is converted into a high-dimensional feature vector through the feature extractor. These vectors serve as nodes in the graph of the GCN, representing individual data points in the feature space. The initial nodes are defined as input image features f_i. The EdgeNet $\mathcal{C}_{\text{edge}}$, parameterized by Θ_E, constructs an affinity matrix E by assessing the pairwise relationships between the feature vectors. For nodes u and v, representing the feature vectors f_u and f_v, respectively, the EdgeNet $\mathcal{C}_{\text{edge}}$ computes the pairwise feature similarities and processes them through its convolutional layers and the sigmoid activation function to produce the edge weights

$$e_{u,v}^{(l)} = \sigma\left(\mathcal{C}_{\text{edge}}\left(\left|f_u^{(l)} - f_v^{(l)}\right|; \Theta_E\right)\right) \tag{2}$$

where the edge weight $e_{u,v}^{(l)}$ represents the semantic similarity score between the features of nodes $f_u^{(l)}$ and $f_v^{(l)}$ in the lth layer of GCN. These edge weights are then normalized across all N edges emanating from node $u^{(l)}$ to form a probability distribution representing the connection strengths to neighboring nodes

$$\hat{e}_{u,v}^{(l)} = \frac{e_{u,v}^{(l)}}{\sum_{i=1}^{N} e_{u,i}^{(l)}}. \tag{3}$$

Using the refined edge weights, the NodeNet $\mathcal{C}_{\text{node}}$, with parameters Θ_N, aggregates the features from the neighboring nodes. The aggregation at iteration l for node u is given by

$$a_u^{(l)} = \sum_{v \in \mathcal{N}(u)} \hat{e}_{u,v}^{(l)} h_v^{(l)} \tag{4}$$

where $h_v^{(l)}$ represents the feature vector of node v, and $\mathcal{N}(u)$ denotes the neighborhood of node u. The aggregated features $a_u^{(l)}$ are then concatenated with the node's current features and transformed through NodeNet's operations

$$h_u^{(l+1)} = \sigma\left(W^{(l)} \cdot \left[h_u^{(l)} \| a_u^{(l)}\right] + b^{(l)}\right) \tag{5}$$

where $\|$ denotes concatenation operation, $W^{(l)}$ and $b^{(l)}$ are the weights and bias at iteration layer l of the NodeNet, and $\sigma(\cdot)$ is a nonlinear activation function.

During the process of collaborative learning, the MLP classifier $\mathcal{C}_{\text{mlp}}$, parameterized by Θ_{mlp}, predicts the target labels of each batch through individual feature learning. $\mathcal{C}_{\text{mlp}}$ provides instance-level predictions, which are less susceptible to noise influence from different neighbors. The predictions are utilized to supervise the $\mathcal{C}_{\text{edge}}$, ensuring that the learned edge weights accurately reflect the true relationships between samples. This is achieved by constructing an affinity matrix, where each element $\tilde{\epsilon}_{u,v}$ is

$$\tilde{\epsilon}_{u,v} = \begin{cases} 1, & \text{if } \tilde{y}_u = \tilde{y}_v \\ 0, & \text{otherwise} \end{cases} \tag{6}$$

where $\tilde{y}_u$ and $\tilde{y}_v$ are the labels corresponding to the input images x_u and x_v, respectively. Here, $\tilde{y}_i = \{\tilde{y}_i^s, \tilde{y}_i^t\}$ and $x_i = \{x_i^s, x_i^t\}$. For the labeled data x_i^s from the source domain in the cloud device, the corresponding label $\tilde{y}_i^s$ represents the ground truth y_i^s of the sample. For unlabeled target domain data x_i^t from various terminals, $\tilde{y}_i^t$ is represented as the predicted label of $\mathcal{C}_{\text{mlp}}$. To ensure the accuracy of the supervision, DCCL determines whether the confidence entropy of the samples exceeds a given threshold τ_s. The confidence entropy is calculated based on the probability vectors outputted by the softmax layer in the MLP classifier. For the sample whose confidence entropy does not exceed the threshold τ_s, the corresponding edge weights are masked and not optimized during the training of the current batch. The confidence entropy is calculated as

$$\mathcal{Q}_{\text{thr}}(\hat{p}_c) = 1 - \sum_{c=1}^{C} \frac{\hat{p}_c \log \hat{p}_c}{\log 1/C} \tag{7}$$

where $\hat{p}_c$ is the probability vector, and C is the number of classes. The value of $Q_{\text{thr}}(\cdot)$ ranges from 0 to 1, and it close to 1 indicates the model is confident enough for the prediction.

Meanwhile, based on the dual classifier structure of the DCCL module, GCN achieves more robust semantic-level feature aggregation by learning the mutual relationships between nodes. Therefore, DCCL combines an adaptive mechanism, merging the target domain data predicted by the $\mathcal{C}_{\text{node}}$ with high-confidence pseudo-labels into the source domain. Specifically, at each iteration iter, the model assigns a pseudo-label $\bar{y}_i^t$ to data from the target domain if the confidence entropy exceeds the threshold τ_s

$$\bar{y}_i^t = \begin{cases} \underset{c' \in \mathcal{C}}{\text{argmax}} \ \hat{p}\left(\bar{y}_i^t = c' | \Theta_{\text{node}}\right), & \text{if } \mathcal{Q}_{\text{thr}}(\hat{p}_{c'}) > \tau_s \\ \text{unlabeled}, & \text{otherwise.} \end{cases} \tag{8}$$

Therefore, the evolving source dataset during the adaptation process is represented as

$$\mathcal{D}_s^{(\text{iter}+1)} = \mathcal{D}_s^{(\text{iter})} \cup \left\{\left(x_i^t, \bar{y}_i^t\right) | \ \bar{y}_i^t \text{ is labeled}\right\} \tag{9}$$

which includes the original source data $\mathcal{D}_s$ and the pseudo-labeled target data from $\mathcal{D}_t^k$ that is deemed reliable. Note that $\mathcal{D}_s^{(0)} = \{\mathcal{D}_s\}$ means that only labeled source domain data is used in the early iter for more reliable training. This evolving dataset serves as the ground-truth feedback for training the MLP classifier across various target domains. It plays a significant role in bridging the domain gap and gradually adapting semantic-level features, leading to a more accurate cloud model.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

After completing the centralized learning stage and obtaining a robust cloud network, we use the GCN classifier to generate pseudo-labels for each target domain under the confidence threshold τ_t, facilitating the training of the second personalized fine-tuning stage.

C. HNP Module

During the centralized learning stage, the DCCL module identifies a shared feature space across multiple terminal data domains for feature extraction. In the personalized fine-tuning stage, TeCCo aims to finely adjust lightweight models on terminals so that they can focus on specific subspaces corresponding to local data distribution, thereby further enhancing the real-time interpretation accuracy of terminals. To facilitate this, we introduce the HNP module, which serves as an auxiliary neural network tasked with producing a personalized set of parameters Θ^k_{mlp} for each terminal's MLP classifier $\mathcal{T}^k_{\text{mlp}}$, where $k = 1, 2, \ldots, N_t$ represents the kth terminal among a total of N_t terminals. The HNP module allows efficient end-to-end training with fewer network parameters through backpropagation. As a form of flexible parameter sharing, HNP balances cloud-based sharing with terminal-specific customization, effectively tackling the challenge of domain-specific classification within a unified model architecture.

First, for the cloud dataset $\mathcal{D}_s$ and each terminal dataset $\mathcal{D}^k_t$, their corresponding domain ID is represented by a one-hot vector d_k, and note that we set $k = 0$ represents the cloud ID d_0 to unify the expression. The setting of a one-hot vector ensures different domains do not share dimensions during training. This means each domain has its own independent representation space, aiding the model to understand differences between domains and better achieve personalization. We denote the HNP module as $\mathcal{H}(\omega)$, where ω encapsulates the parameters of the hypernetwork. $\mathcal{H}(\omega)$ takes the received d_k and transforms it into a specific embedding representation $\epsilon_i = f(d_k)$, where $f : \mathbb{R}^{N_{d_k}} \to \mathbb{R}^{N_e}$ is a deterministic function mapping the domain ID to a N_e dimensional space. Second, for each domain-specific input ϵ_i, the HNP module generates an output through a sequence of affine transformations followed by nonlinear activation functions across its layers. At the final layer of the HNP, we obtain the output vector which contains all parameters for the MLP in a serialized form.

Formulaically, the final output of the hypernetwork for the kth domain is given by

$$\mathbf{v}_{d_k} = W_{\text{out}} \cdot r'_{d_k} + B_{\text{out}} \tag{10}$$

where W_{out} is the weight matrix for the output layer of the hypernetwork, r'_{d_k} is the activation from the last hidden layer, and B_{out} is the bias vector for the output layer. The serialized parameter vector $\mathbf{v}_{d_k}$ is partitioned and transformed into the necessary set of weight matrices and bias vectors for each layer of the MLP classifier. For the MLP classifier containing L layers, the lth layer utilizes a weight matrix $W_{il} \in \mathbb{R}^{d_l \times d_{l-1}}$ and a bias vector $b_{il} \in \mathbb{R}^{d_l}$. Here, d_l represents the number of neurons in layer l, and d_{l-1} indicates the number of neurons in the previous layer. The reshaping process is defined as follows:

$$W_{il} = \text{Mat}\left(\mathbf{v}_{d_k}[\alpha_l : \alpha_l + d_l \cdot d_{l-1}]\right) \tag{11}$$

$$b_{il} = \mathbf{v}_{d_k}[\alpha_l + d_l \cdot d_{l-1} : \alpha_l + d_l \cdot d_{l-1} + d_l] \tag{12}$$

where Mat is an operator that reshapes a segment of the vector into a matrix of specified dimensions, and α_l denotes the cumulative index that marks the beginning of the parameters for the lth layer within the vector $\mathbf{v}_{d_k}$. Therefore, the complete set of parameters Θ_k for the MLP classifier corresponding to the kth terminal model is expressed as

$$\Theta^k_{\text{mlp}} = \{(W_{i1}, b_{i1}), (W_{i2}, b_{i2}), \ldots, (W_{iL}, b_{iL})\} \tag{13}$$

where each tuple (W_{il}, b_{il}) contains the weight matrix and bias vector for the lth layer of the classifier.

It is worth noting that during the centralized learning stage, in order to enable the cloud model to learn data features from all domains, the domain ID of all training data is uniformly represented by the cloud ID d_0. They are input into the HNP module to generate the cloud $\mathcal{C}_{\text{mlp}}$ parameters. In the personalized fine-tuning stage, the linear layer used to generate classifier parameters in the HNP module is frozen. The only component that requires training is the embedding layer, responsible for transforming each terminal domain ID d_k into a feature vector. This approach allows for an efficient search for personalized parameters Θ^k_{mlp}, making them more suitable for each terminal classifier $\mathcal{T}^k_{\text{mlp}}$ within the overall optimization space.

D. Data-Assisted Progressive Inference Mechanism

Progressive inference offers a rapid and dependable CNN inference mechanism that operates across both terminals and the cloud [67], [68]. This method is particularly well-suited for a distributed framework based on the layer-splitting model, where shallow layers deployed at the terminal are permitted to exit the network early upon reaching a specified threshold. As a result, a balance between efficiency and accuracy is achieved for the terminal-cloud framework. However, when dealing with a broader application scenario, where terminal and cloud models are relatively distinct and independent, existing progressive inference approaches can only hope for a limited and fixed accuracy improvement by capitalizing on the feature extraction capability of the cloud model.

To address this limitation, we propose a data-assisted progressive inference mechanism that ingeniously combines the clustering capabilities of the $\mathcal{C}_{\text{node}}$ in GCN classifiers within the DCCL module. This data-assisted mechanism not only employs the cloud feature extractor to capture more robust features, but also incorporates labeled source domain data from the cloud as supplementary guidance. By leveraging the available information in the cloud effectively, the data-assisted approach empowers the progressive inference mechanism to optimize efficiency through early terminal exits while achieving more substantial accuracy enhancements in the cloud.

The data-assisted progressive inference process is shown in Fig. 2, where each terminal collects data and first uses the local model for inference. The terminal classifier $\mathcal{T}^k_{\text{mlp}}$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

obtains the probability vector p_i after each input image x_i passes through the softmax layer and calculates the normalized confidence entropy according to (7). If the confidence entropy exceeds the given threshold τ_p, it indicates that the terminal is confident enough to directly output the result. Otherwise, the initially processed features from the terminal are transmitted to the cloud device. These features, along with the features of labeled data from the source domain, are input into $\mathcal{C}_{\text{node}}$ to achieve more accurate clustering results. This approach is based on our observation during the training of the DCCL module, where the $\mathcal{C}_{\text{node}}$ classifier, supervised by the $\mathcal{C}_{\text{mlp}}$ classifier, demonstrated superior clustering capabilities. Therefore, we leverage labeled data from the source domain to assist the clustering process. It is worth noting that in order to make better use of the source domain data in the cloud for supervised clustering, it is necessary to accurately align the features extracted from the cloud model with those from the terminal model. Therefore, during the centralized learning stage, a distillation loss is designed as the key to achieving feature alignment, enabling joint training of the cloud model and the generic terminal model. This generic terminal model will serve as the initial weight for each terminal in the personalized fine-tuning stage.

E. Pipeline and Loss Optimization

This section provides a detailed explanation of the loss functions and optimization processes for the TeCCo framework. The pipeline is illustrated in Algorithm 1. The optimization aims to enhance the cloud model's generalization capabilities while ensuring that each terminal model achieves a high-precision interpretation of the corresponding local data. During the initial centralized learning phase, each iteration iter involves sampling a batch of samples $\mathcal{B}$ from the dataset $\mathcal{D}_s^{(\text{iter})}$. The update via $\mathcal{D}_s^{(\text{iter})}$ is described following (9), and $\mathcal{D}_s^{(\text{iter})}$ contains all the source domain data with ground truth y^s, as well as some target domain data from multiple terminals for which the node network $\mathcal{C}_{\text{node}}$ is already confident enough to classify, and the predicted label by $\mathcal{C}_{\text{node}}$ is denoted as $\bar{y}^t$. For concise expression, we uniformly represent the label of samples in $\mathcal{B}$ as $\bar{y}^* = \{y^s, \bar{y}^t\}$. Then, these samples $\mathcal{B}$ are simultaneously input into the cloud feature extractor $\mathcal{F}_{\text{cld}}$ and the comparatively lightweight terminal feature extractor $\mathcal{F}_{\text{tr}}$. $f_{\text{cld}} = \mathcal{F}_{\text{cld}}(x)$ and $f_{\text{tr}} = \mathcal{F}_{\text{tr}}(x)$ represent the feature maps of sample x after passing through two feature extractors, respectively. We introduce a feature-based knowledge distillation guided by mse loss to minimize the gap in representations between the cloud and terminal models, thereby guiding the lightweight feature extractor at the terminal to learn from the cloud model. The distillation loss formula is calculated as

$$\mathcal{L}_{\text{distill}} = \frac{1}{\mathcal{B}} \sum_{x \in \mathcal{B}} \| f_{\text{cld}} - \mathcal{W}(f_{\text{tr}}) \|_2^2. \tag{14}$$

Here, $\mathcal{W}$ is the linear layer used to align the dimensions of f_{cld} and f_{tr}. The $\mathcal{L}_{\text{distill}}$ naturally treats the cloud-terminal pair as a teacher–student model, facilitating cross-model knowledge transfer during centralized training. Moreover, compared to only training the cloud model first and then fine-tuning each terminal model using generated pseudo-labels, incorporating feature-level distillation loss for joint training of the cloud model and generic lightweight terminal model offers significant advantages. On one hand, it can substantially reduce the time required for the personalized fine-tuning phase, thereby accelerating convergence. On the other hand, it helps the terminal model learn feature representations aligned with the cloud model. Consequently, during the progressive inference, the cloud model can not only further process the features handled by the terminal but also leverage features from source domain data on the cloud, enabling more robust clustering by combining the node classifier.

Subsequently, f_{cld} is input to the DCCL module and fed into the HPM and the GCN simultaneously. HPM also receives the domain ID d_0 corresponding to the cloud, and outputs the parameters of the MLP classifier $\mathcal{C}_{\text{mlp}}$ in this phase. The MLP classifier outputs a probability distribution $\tilde{z}_i$ and the predicted class label $\tilde{y}_i$ for each input image $x_i \in \mathcal{B}$, which is obtained through the logits generated by the softmax function. Therefore, the cross-entropy loss function of the

Algorithm 1 Pipeline for TeCCo

Require: Source domain data $\mathcal{D}_s$, Target domain data $\mathcal{D}_t^k$,
Ensure: Trained cloud model, Personalized terminal models

Centralized Learning Stage:
Initialize cloud feature extractor $\mathcal{F}_{cld}$ and generic terminal feature extractor $\mathcal{F}_{tr}$
while centralized learning not converged **do**
  Sample batch $\mathcal{B}$ from $\mathcal{D}_s^{(iter)}$
  Extract features $f_{cld} = \mathcal{F}_{cld}(x)$ and $f_{tr} = \mathcal{F}_{tr}(x)$ for $x \in \mathcal{B}$
  Compute distillation loss $\mathcal{L}_{distill}$ (14)
  Input f_{cld} to DCCL module:
    Compute MLP classifier loss $\mathcal{L}_{mlp}$ (15)
    Compute EdgeNet loss $\mathcal{L}_{edge}$ (16)
    Compute NodeNet loss $\mathcal{L}_{node}$ (17)
  Compute adversarial loss $\mathcal{L}_{adv}$ (18)
  Update model parameters by minimizing the total loss $\mathcal{L}_{total}$ (19)
end while
Personalized Fine-Tuning Stage:
for each terminal k **do**
  Initialize the terminal feature extractor using the weights $\mathcal{F}_{tr}$ trained in the first stage
  Generate high-confidence pseudo labels for local data $\mathcal{D}_t^k$ using the cloud model
  Fine-tune terminal model $\mathcal{T}_{mlp}^k$ on local data $\mathcal{D}_t^k$ using cross-entropy loss
end for
Progressive Inference Mechanism:
for each terminal k during inference **do**
  Perform local inference on terminal data
  if prediction confidence is low **then**
    Transfer features to cloud for accurate processing
  end if
end for =0

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

MLP classifier for a single sample x_i can be expressed as

$$\mathcal{L}_{\text{mlp}} = -\frac{1}{\mathcal{B}} \sum_{i=1}^{\mathcal{B}} \bar{y}_i^* \log p(\tilde{y}_i | \tilde{z}_i). \tag{15}$$

For GCN, the EdgeNet $\mathcal{C}_{\text{edge}}$ encourages the model to identify and reinforce the similarities among the features of instances from the same class, and weakens the connections between instances from different classes. In the DCCL module, the high-confidence MLP classification labels supervise the weights of the edge affinity matrix using binary cross-entropy, as shown in (6). The computation of the EdgeNet loss between samples x_u and x_v is given by

$$\mathcal{L}_{\text{edge}} = \tilde{\epsilon}_{u,v} \log p(\hat{e}_{u,v}) + (1 - \tilde{\epsilon}_{u,v}) \log(1 - p(\hat{e}_{u,v})). \tag{16}$$

Here, $\hat{e}_{u,v}$ is the normalized edge weight. Subsequently, the node network $\mathcal{C}_{\text{node}}$ aggregates neighboring features according to (4) to achieve more robust classification. Similarly, it outputs a probability distribution $\bar{z}_i$ and the classification loss is calculated using cross-entropy

$$\mathcal{L}_{\text{node}} = -\frac{1}{\mathcal{B}} \sum_{i=1}^{\mathcal{B}} \bar{y}_i^* \log p(\bar{y}_i | \bar{z}_i). \tag{17}$$

Furthermore, TeCCo leverages the concept of domain adversarial neural network [69], employing a discriminator to align the feature distributions of the source domain and various target domains across the cloud and terminals. In the centralized learning stage, the feature extractor $\mathcal{F}_{\text{cld}}$ and the discriminator D are involved, where $\mathcal{F}_{\text{cld}}$ attempts to produce domain-invariant features, and D tries to distinguish the domain of origin from the extracted features. Considering all terminal data as a unified target domain, the adversarial discriminator loss is formulated as

$$\begin{aligned}\mathcal{L}_{\text{adv}} = &-\frac{1}{\mathcal{B}_s} \sum_{x_i^s \sim \mathcal{B}_s} \log\left[D\left(\mathcal{F}_{\text{cld}}\left(x_i^s\right)\right)\right] \\ &- \frac{1}{\mathcal{B}_t} \sum_{x_i^t \sim \mathcal{B}_t} \log\left[1 - D\left(\mathcal{F}_{\text{cld}}\left(x_i^t\right)\right)\right]\end{aligned} \tag{18}$$

where $\mathcal{B}_s \in \mathcal{D}_s$ and $\mathcal{B}_t \in \mathcal{D}_t^k$ represent the source domain and target domain data, respectively. Therefore, during the centralized learning stage, the overall optimization objective of TeCCo can be formalized as

$$\begin{aligned}\mathcal{L}_{\text{total}} = &\lambda_{\text{mlp}}\mathcal{L}_{\text{mlp}} + \lambda_{\text{edge}}\mathcal{L}_{\text{edge}} + \lambda_{\text{node}}\mathcal{L}_{\text{node}} \\ &+ \lambda_{\text{distill}}\mathcal{L}_{\text{distill}} - \lambda_{\text{adv}}\mathcal{L}_{\text{adv}}\end{aligned} \tag{19}$$

where λ_{mlp}, λ_{edge}, λ_{node}, λ_{distill}, and λ_{adv} are the weight factors of different loss functions.

In the personalized fine-tuning stage, the HPM assigns specific parameters to each terminal classifier $\mathcal{T}_{\text{mlp}}^k$ based on its domain ID. Simultaneously, the robust cloud model, after centralized training, can generate high-confidence pseudo labels for the target domain in each terminal, thereby assisting in the fine-tuning of terminal models to enhance accuracy. It is important to note that a generic lightweight terminal feature extractor $\mathcal{F}_{\text{tr}}$ has been trained using distillation loss during the centralized learning stage. Consequently, the personalized fine-tuning stage demands only minimal training time and computational resources. The fine-tuning loss for the terminal setup is represented by the cross-entropy loss. Subsequent experiments examined the impact of fine-tuning strategy on the terminal's performance.

IV. Experiment

In this section, we conduct comprehensive experiments to demonstrate the superiority of TeCCo. We begin by describing the experimental settings, followed by a comparison of TeCCo with several state-of-the-art (SOTA) algorithms. Various evaluation metrics are employed to showcase TeCCo's benefits from multiple perspectives. In addition, ablation studies are performed to underscore the importance of each module within TeCCo. Finally, we present a visual analysis to clearly illustrate the effectiveness of TeCCo and perform a computational cost analysis to assess its practical applicability.

A. Setting Up

1) Dataset: We evaluate TeCCo using five public RS datasets: AID [70], NWPU-RESISC45 [71], PatternNet [72], RSD46-WHU [73], [74], and UCMerced [75]. The detailed description of these five datasets are as follows.

a) AID: It is a large-scale dataset designed for aerial scene classification. All images are meticulously labeled by RS image interpretation specialists. AID's multisource, multiresolution images result in higher intraclass variations and smaller interclass dissimilarities.

b) NWPU-RESISC45: It is a public benchmark known as the RS Image Scene Classification (RESISC), developed by Northwestern Polytechnical University (NWPU). The images in this dataset demonstrate variations in different aspects, including translation, spatial resolution, and illumination, resulting in significant within-class diversity and between-class similarity.

c) PatternNet: It is collected from Google Earth imagery and Google Maps, representing numerous US cities at diverse spatial resolutions. The images encompass resolutions ranging from 0.062 to 4.693 m, resulting in high intraclass diversity and notable interclass similarity.

d) RSD46-WHU: The dataset is derived from Google Earth imagery and Tianditu, sourced from sensors involving panchromatic and multispectral data. Most classes have a ground resolution of 0.5 m, while others are approximately 2 m, presenting comprehensive performance challenges due to the diversity of the dataset.

e) UCMerced: This dataset was obtained from the United States Geological Survey National Map. Some images exhibit homogeneity in texture or color, while others present a mix of varied spatial characteristics, thereby providing an intriguing challenge for research and analysis.

We utilize five datasets to represent the data accumulated from different terminals, each corresponding to a unique domain. From the detailed statistics provided in Table I, it is apparent that these datasets exhibit diverse class distributions. To enable algorithms to effectively learn across different domains, it is necessary to standardize these datasets

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DETAILED STATISTICS AMONG FIVE ORIGINAL DATASETS

DataSet	# Image	# Class	# Image per Class
AID	10,000	30	220-420
NWPU-RESISC45	31,500	45	700
PatternNet	30,400	38	800
RSD46-WHU	117,000	46	500-3000
UCMerced	2100	21	100

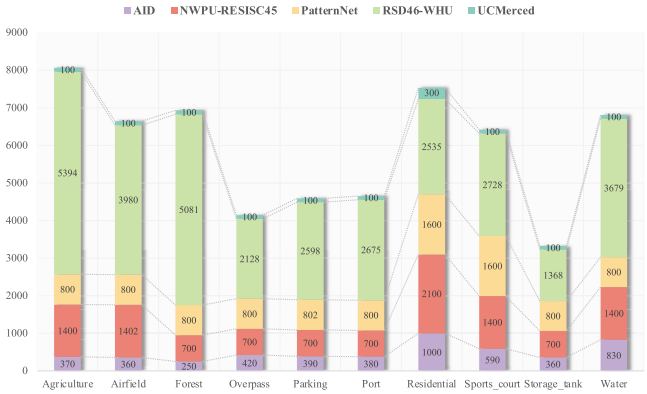


Fig. 3. Comprehensive statistics for the five adjusted datasets.

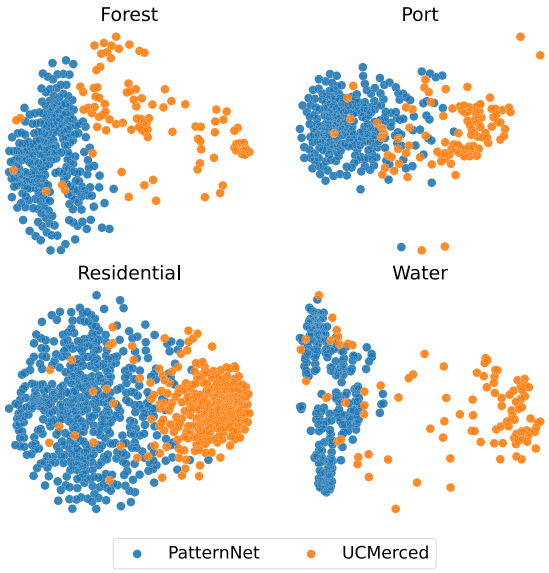


Fig. 4. Illustration of t-SNE applied to the forest, port, residential, and water categories from PatternNet and UCMerced datasets.

to ensure each one incorporates the same classes. In accordance with [21], we combine several classes into broader categories. For example, within PatternNet, the classes "baseball_field" and "football_field" are consolidated into the category sports_court. This approach ensures that all five datasets consist of the same ten classes: agriculture, airfield, forest, overpass, parking, port, residential, sports_court, storage_tank, and water.

The data distribution for each class across five datasets is depicted in Fig. 3. As observed, there is an imbalance in the number of images derived from different datasets. This imbalance presents a challenge for model training but also provides an opportunity to emulate real-world scenarios more accurately. In real-world applications, different terminals capture varying amounts of RS images from their respective domains. Hence, this dataset disparity could potentially lead to more realistic model performance.

To illustrate the differences across various domains, we employ t-SNE [76] to visualize images from the same categories but different datasets, as depicted in Fig. 4. We randomly sample images from the forest, port, residential, and water categories in the PatternNet and UCMerced datasets. The notable disparities within the same category across different datasets suggest that a pretrained model from one dataset may not perform effectively when applied to another. Therefore, we propose the novel TeCCo to mitigate the challenge.

2) Evaluation Protocol: In order to mirror real-world scenarios where various terminals collect diverse, unlabeled RS images, we adhere to standard training and testing protocols of domain adaption learning to conduct the experiments. To be specific, we utilize one dataset as the source domain, which contains ground-truth labels, and four other datasets as the target domains without labels. Each dataset is sequentially given the opportunity to serve as the source domain.

Based on the distributed framework of TeCCo, we utilize several evaluation metrics to comprehensively access its advantages, which are described as follows.

a) Terminal accuracy: We utilize this metric to scrutinize the performance of the classifier on the terminal device. We denote this evaluation metric as Acc^{T}, serving as a critical metric for comparison against baselines.

b) Cloud MLP accuracy: This metric evaluates the MLP classifier on the cloud. It can be estimated in two ways. First, by using the entire model on the cloud to process images from different domains, denoted as Acc^{CM}. Second, by using only the MLP classifier to process features transmitted from terminals in the data-assisted progressive inference mechanism, denoted as Acc^{PCM}.

c) Cloud GCN accuracy: This metric is similar to cloud MLP accuracy but specifically assesses the GCN classifier on the cloud. Like the MLP classifier, the performance of the GCN classifier can also be estimated in two ways, denoted as Acc^{CG} and Acc^{PCG}, respectively.

d) Progressive ratio: We employ this specific metric to record the ratio of data transmitted to the cloud when applying the data-assisted progressive inference mechanism. Notably, a lower value for this metric indicates the superior performance of the terminal classifier, as fewer data require cloud-side intervention for accuracy enhancement.

e) Confusion matrix: This serves as a specific table layout allowing visualization of the performance. It provides a more nuanced understanding of performance, including insights into precision and recall, and helps identify any model bias across different categories and domains.

In the experiments, we use terminal accuracy as the primary metric to evaluate the final performance of TeCCo, comparing it against SOTA baselines. This choice is driven by the objective to develop a classifier for each terminal that

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

consistently delivers high-quality performance. Then, we use other metrics to comprehensively demonstrate the advantages of TeCCo from different perspectives.

3) Implementation Details: In the implementation of TeCCo, we deploy the ResNet-50 [77] as the feature network situated in the cloud, while utilizing the lightweight ResNet-18 as the feature network for the terminal. The construction of the hypernetwork incorporates an embedding layer with a size of 128, which is used to process the input domain ID. Subsequently, we employ a fully connected layer equipped with 512 units to manage the resulting embeddings, outputting the parameters for the classifier dedicated to each domain. For the training process, we use an SGD optimizer configured with a learning rate of 0.001 and weight decay of 0.005 and we set the batch size to 32. Regarding the coefficients in (19), we assign $\lambda_{\text{node}} = 0.3$, $\lambda_{\text{edge}} = 0.3$, and set the other terms to 1.0. In addition, we need to set three thresholds τ_s, τ_t, and τ_p in the whole training and inference process. During the model training process on the cloud, we establish a confidence level for setting pseudo labels on the target domain, signified by the parameter $\tau_s = 0.7$. After training the model on the cloud, we employ the GCN classifier to assign pseudo labels for the fine-tuning phase. This process involves maintaining a high confidence threshold $\tau_t = 0.9$. Finally, we carry out data-assisted progressive inference, employing the confidence threshold $\tau_p = 0.7$ to determine which data are eligible for transmission to the cloud. All experiments are conducted on the P40 GPU with PyTorch 1.8.1+cu101 framework and repeated with four random different seeds.

B. Comparative Analysis

To demonstrate the superiority of TeCCo, we conduct comparative analyses against various baselines within the terminal-cloud framework and cross-domain learning schemes. Through addressing the following key questions: 1) Why is the capability to process images from cross-domain necessary in the terminal-cloud framework? 2) How does the collaborative learning of TeCCo perform compared to recent SOTA baselines with adaptation learning schemes?

To address the first question, we compare TeCCo with DCNNet [10], which employs a progressive inference mechanism within the terminal-cloud framework. We conduct this comparison from three perspectives. First, we use the terminal accuracy (Acc^{T}) of TeCCo to compare the performance of each terminal on the target domains with that of DCNNet. Second, we use the cloud accuracy (Acc^{CM}) of TeCCo to compare the performance of the cloud on the target domains with that of DCNNet. These comparisons will demonstrate the advantage of incorporating adaptive learning in the terminal-cloud framework. Finally, we use the progressive accuracy (Acc^{PCG}) of TeCCo to compare the performance of progressive inference in DCNNet, highlighting the advantage of our data-assisted mechanism.

We use the AID, NWPU-RESISC45, and UCMerced as source domains with ground truth labels and evaluate the performance of TeCCo and DCNNet on four target domains, respectively. As shown in Table II, TeCCo outperforms DCNNet from all perspectives, fully demonstrating TeCCo's advantage in processing RS images from cross-domain. DCNNet leverages the differing capacities of neural networks on terminal and cloud devices to achieve high performance. However, despite these varied capacities, the networks are trained on images from the same domain, resulting in homogeneous terminals that struggle to effectively process RS images from different domains. TeCCo utilizes the DCCL and HNP modules to achieve better cross-domain generalization and employs data-assisted progressive inference to enhance performance. This finding underscores the necessity of processing images from cross-domain in real-world applications.

We now investigate the second question by comparing TeCCo with SOTA methods that incorporate cross-domain learning schemes. We utilize Acc^{CM} and Acc^{T} as evaluation metrics and calculate both metrics as the average over target domains. Acc^{CM} highlights the benefits of the DCCL module when using a single model to learn from cross-domain. Acc^{T} is an important evaluation metric because the performance of each terminal is the ultimate objective.

We conditioner baselines as follows. DDC [78] introduces an adaptation layer and an additional domain confusion loss to learn representations. DAN [79] maps task-specific layers into a reproducing kernel Hilbert space, facilitating the matching of mean embeddings of different domains. DANN [69] employs standard layers and a gradient reversal layer to encourage features that are discriminative for the source domain and invariant to domain shifts. Deep Coral [80] learns a nonlinear transformation that aligns the second-order statistics of source and target distributions by matching layer activation correlations. JAN [81] aligns the joint distributions of multiple domain-specific layers across domains using a joint maximum mean discrepancy criterion. CDAN [82] captures the cross-covariance between feature representations, alongside entropy conditioning to control classifier prediction uncertainty. TADA [83] integrates two types of transferable attention from local and global perspectives to enhance domain adaptation learning effectiveness. TSAN [21] employs a two-stage training scheme combining adversarial learning to confound the classifier and self-supervised learning to effectively partition the target domain. OSBP [84] uses adversarial training to extract features that separate unknown target samples from known ones. DAMC [85] introduces a multiclassifier structure to target samples, encouraging positive transfers and reducing the domain gap. BSP [86] employs singular value decomposition to constrain the larger singular values, strengthening other eigenvectors to enhance feature discriminability. MDD [87] introduces margin disparity discrepancy, tailored to distribution comparison using an asymmetric margin loss and minimax optimization for training. AMRAN [88] considers both marginal and conditional distributions for comprehensive alignment, using attention mechanisms and a multiscale strategy to extract robust features. SSOUDA [89] introduces a contrastive self-supervised learning network, along with a novel open-set class learning module to identify reliable unknown class samples. We access the results from [21] as the performance of DDC, DAN, DANN, Deep Coral, CDAN, TADA, and TSAN, and then we replicate the outcomes using the official implementation for other baselines. We also

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON RESULTS BETWEEN TECCO AND DCNNET

AID as source domain with ground truth label						
Dataset		NWPU-RESISC45	PatternNet	RSD46-WHU	UCMerced	Average
Terminal	DCNNet	80.47	85.32	80.09	87.76	83.41
	TeCCo (Acc^{T})	95.78	97.72	93.45	99.08	**96.51**
Cloud	DCNNet	82.63	87.03	83.16	90.02	85.71
	TeCCo (Acc^{CM})	96.05	97.45	91.63	98.62	**95.94**
Progressive	DCNNet	83.13	89.15	84.52	91.84	87.16
	TeCCo (Acc^{PCG})	97.35	97.22	94.98	99.65	**97.30**
NWPU-RESISC45 as source domain with ground truth label						
Dataset		AID	PatternNet	RSD46-WHU	UCMerced	Average
Terminal	DCNNet	87.45	83.11	78.26	81.50	82.58
	TeCCo (Acc^{T})	99.55	95.27	91.03	99.42	**96.32**
Cloud	DCNNet	88.09	84.68	80.35	82.26	83.35
	TeCCo (Acc^{CM})	98.28	94.38	88.29	98.54	**94.87**
Progressive	DCNNet	89.75	85.43	82.74	84.85	85.69
	TeCCo (Acc^{PCG})	99.51	95.43	90.98	99.65	**96.39**
UCMerced as source domain with ground truth label						
Dataset		AID	PatternNet	RSD46-WHU	NWPU-RESISC45	Average
Terminal	DCNNet	81.20	77.05	65.25	65.45	72.24
	TeCCo (Acc^{T})	94.14	99.89	83.75	83.34	**90.28**
Cloud	DCNNet	82.56	79.82	70.10	66.13	74.65
	TeCCo (Acc^{CM})	90.92	98.51	79.17	81.31	**87.48**
Progressive	DCNNet	85.02	81.34	75.94	71.75	78.51
	TeCCo (Acc^{PCG})	94.29	99.93	83.70	84.15	**90.52**

TABLE III
COMPARED RESULTS WITH OTHER METHODS

Methods	A → N, P, R, U	N → A, P, R, U	P → A, N, R, U	R → A, N, P, U	U → A, N, P, R	Average
Baseline	75.98	70.79	45.33	61.12	58.07	62.26
DCC [78]	79.23	74.45	51.06	66.70	61.97	66.68
DAN [79]	71.55	70.13	59.44	65.58	58.89	65.12
DANN [69]	77.34	73.50	58.41	68.85	61.89	68.00
Deep Coral [80]	77.94	74.47	51.91	67.49	59.97	66.36
JAN [81]	72.03	66.95	59.93	62.37	62.90	64.84
CDAN [82]	80.25	72.87	52.68	67.80	63.30	67.38
TADA [83]	81.26	72.32	55.03	63.98	51.79	64.88
TSAN [21]	84.25	81.87	60.52	73.41	68.10	73.63
OSBP [84]	80.38	80.23	71.59	64.05	71.76	73.60
DAMC [85]	74.79	77.47	67.27	65.37	67.54	70.49
BSP [86]	93.02	93.49	78.82	**91.31**	81.75	87.68
MDD [87]	92.59	91.61	69.92	88.50	77.24	83.97
AMRAN [88]	85.00	86.21	63.45	73.35	66.98	75.00
SSOUDA [89]	76.28	85.02	67.40	68.23	67.82	72.95
TeCCo (Acc^{CM})	**95.94**	**94.87**	**80.51**	91.06	**87.48**	**89.97**
TeCCo (Acc^{T})	**96.51**	**96.32**	**84.09**	**93.64**	**90.28**	**92.17**

integrate a fundamental method called Baseline that utilizes ResNet-50 as the feature network, which is followed by a singular MLP classifier.

The comparative results are delineated in Table III. We adopt capital initials as dataset representations. As an illustration, the label "A → N, P, R, U" implies the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

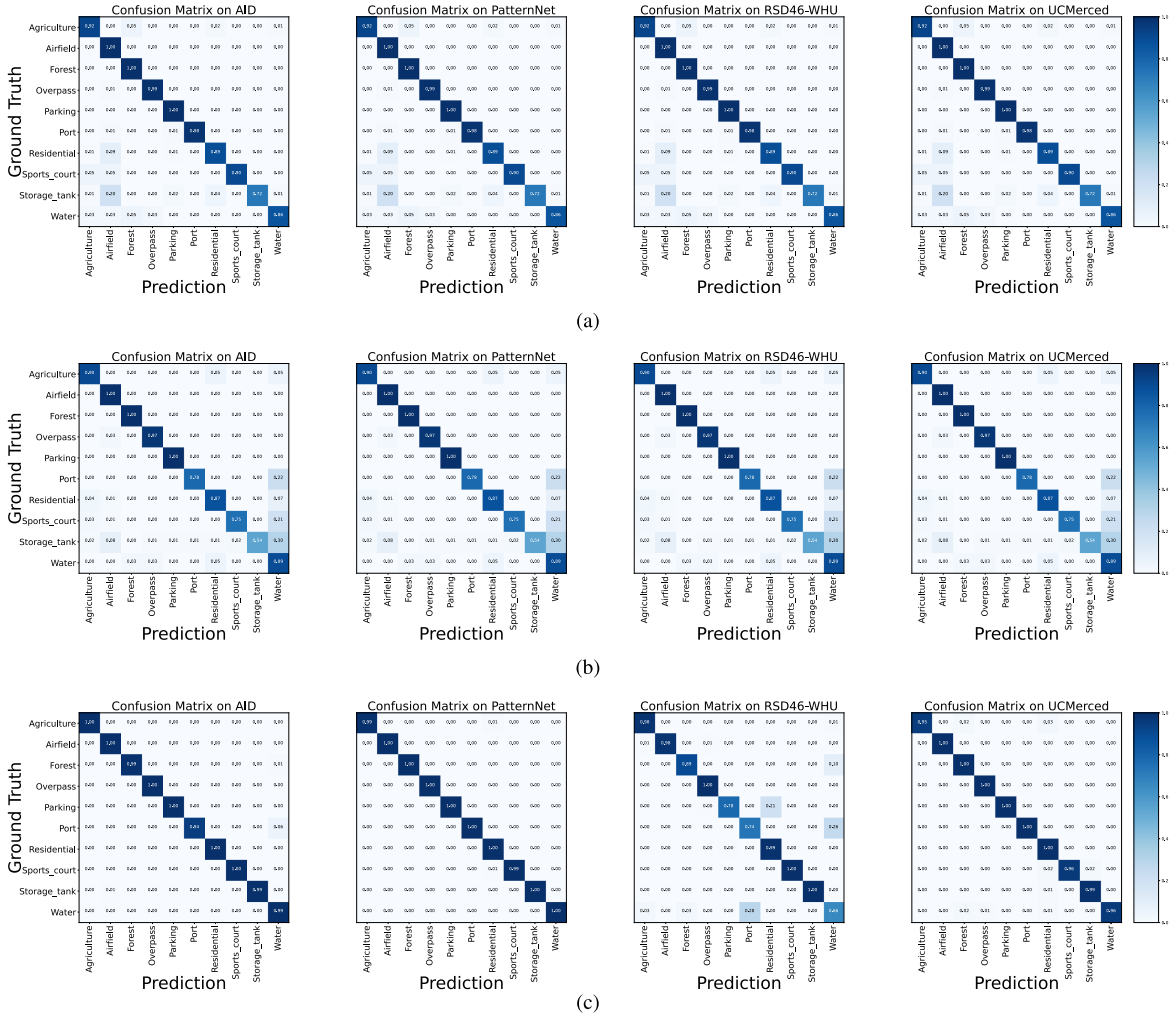


Fig. 5. Confusion matrix of different algorithms when NWPU-RESISC45 as source domain. (a) AMRAN. (b) SSOUDA. (c) TeCCo.

employment of the AID dataset as the labeled source domain in the cloud, while NWPU-RESISC45, PatternNet, RSD46-WHU, and UCMerced are utilized as unlabeled target domains from each terminal.

From the experimental results, it is clear that TeCCo consistently outperforms other methods on both the cloud-side and terminal-side, delivering the highest performance across all settings. Notably, TeCCo achieves exceptional outcomes in the "A $\rightarrow$ N, P, R, U" and "N $\rightarrow$ A, P, R, U" settings. This can be attributed to the high-resolution images provided by the AID and NWPU-RESISC45 datasets. Furthermore, TeCCo maintains commendable performance in the "U $\rightarrow$ A, N, P, R" setting, despite the limited data with ground-truth labels from the UCMerced dataset. This robust performance further underscores the resilience of TeCCo under various conditions.

We utilize the confusion matrix to further illustrate the superior performance of TeCCo across each class, as shown in Fig. 5. The NWPU-RESISC45 dataset is chosen as the source domain. In the confusion matrix, a deeper color signifies better performance. The results indicate that TeCCo outperforms other methods across all ten categories. Specifically, TeCCo achieves nearly 100% accuracy across all ten categories in the target domains AID, PatternNet, and UCMerced. Although there are several mispredictions in the target domain RSD46-WHU, TeCCo still significantly outperforms the other methods. The relatively low performance in the RSD46-WHU is likely due to its larger number of images, suggesting that more training epochs on this domain may improve performance.

C. Additional Results on MRSSC2.0

To further demonstrate the scalability of TeCCo, we evaluate it on another cross-domain RS dataset, MRSSC2.0 [90]. This dataset comprises 26 710 images categorized into seven typical scenes across four distinct domains: visible near-infrared (VIS), short-wavelength infrared (SWI), thermal infrared (INF), and synthetic aperture radar (SAR). In these experiments, we include a new baseline, CACDO [91], and obtain its performance results from its original paper. In addition, we reference the results for DAN, JAN, DANN, CDAN, BSP, and MDD from [90], and leverage the open-source implementations to obtain results for other baselines. As shown in Table IV, TeCCo achieves the best average performance on both cloud and terminal environments, further demonstrating its effectiveness in cross-domain RS image classification.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARED RESULTS UNDER DATASET MRSSC2.0

Methods	VIS → SWI	VIS → INF	VIS → SAR	Average
DAN	90.60	74.02	73.67	79.43
JAN	70.29	61.94	74.57	68.27
DANN	80.51	65.89	**89.85**	78.75
CDAN	88.28	71.19	88.86	82.78
BSP	83.22	65.14	88.58	79.65
MDD	88.20	71.39	88.79	82.79
CACDO	92.30	76.70	88.70	85.23
OSBP	85.24	71.26	70.36	75.62
DAMC	89.23	69.54	73.36	77.38
AMRAN	91.82	75.86	61.68	76.45
SSOUDA	90.34	79.86	77.77	82.66
TeCCo (Acc^{CM})	**93.47**	**80.91**	85.32	**86.57**
TeCCo (Acc^{T})	**93.71**	**81.45**	86.84	**87.33**

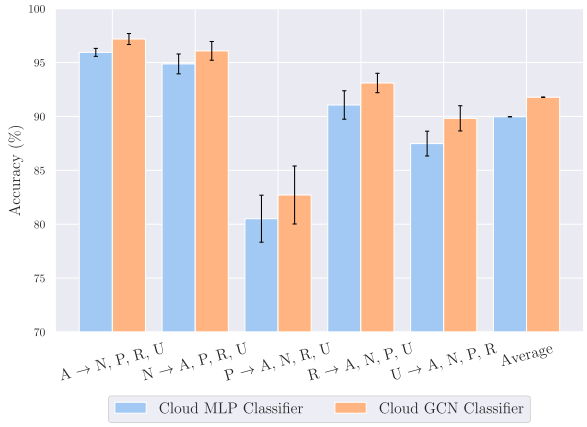


Fig. 6. Performance of MLP and GCN classifiers on a cloud device.

D. Comprehensive Analysis on TeCCo

We employ additional evaluation metrics, previously introduced in Section IV-A, to comprehensively analyze TeCCo.

1) Analysis on DCCL Module: The DCCL module is a crucial component of TeCCo. We design the MLP and GCN classifiers to collaboratively learn knowledge from diverse domains. We estimate the performance of both classifiers by calculating the average Acc^{CM} and Acc^{CG} across all target domains, as shown in Fig. 6. Both the MLP and GCN classifiers demonstrate satisfactory performance. Notably, the GCN classifier outperforms the MLP classifier, which can be attributed to the GCN classifier's ability to extract more informative features. This finding validates TeCCo's effectiveness in delivering strong results for targeted domains, due to the high quality of pseudo-labels generated by the GCN classifier.

To confirm this, we assess the results of the pseudo-label assignment process. We set the confidence threshold $\tau_t = 0.9$ when assigning pseudo-labels to the target domain. First, we calculate the *Ratio* of pseudo-labels that the GCN classifier could provide under this specific confidence threshold. Then, we assess the *Accuracy* of these pseudo-labels, as shown in Table V. Our observations indicate that the GCN classifier can generate pseudo-labels for over 85% of the images from the target domains. The accuracy of these pseudo-labels is notably high, exceeding 95%. In particular, when AID is employed as the source domain, the accuracy of the pseudo-labels approaches nearly 100%, which corresponds with the high Acc^{T} observed in Table III. These results underscore the capability of the GCN classifier to reliably produce high-quality pseudo-labels for each terminal.

TABLE V
EVALUATION ON PSEUDO LABELS PROVIDE BY GCN CLASSIFIER

	A → N, P, R, U	N → A, P, R, U	P → A, N, R, U
Ratio	90.92	92.67	80.76
Accuracy	99.48	98.71	91.73
	R → A, N, P, U	U → A, N, P, R	Average
Ratio	84.52	83.21	86.42
Accuracy	98.59	96.24	96.95

TABLE VI
TRAINABLE PARAMETERS ON FINE-TUNE STAGE FOR EACH CLASSIFIER

Network	hypernetwork	multiple MLP classifiers
Parameters	128	2570

2) Efficiency of HNP: To achieve high-quality performance for each terminal-side classifier, we fine-tune the model using pseudo labels provided by the cloud. This necessitates building a distinct classifier for each terminal. A naive solution involves creating a separate MLP classifier for each terminal. However, to save computational costs and improve performance, we use a hypernetwork to generate the parameters for each MLP classifier, rather than initializing the entire MLP classifier. As shown in Table VI, using the hypernetwork requires training only 128 parameters (one column of the embedding layer) per classifier, compared to 2570 parameters needed when initializing the entire MLP classifier. This finding justifies that using hypernetworks can significantly improve training efficiency during the fine-tuning phase. Furthermore, the hypernetwork can also enhance performance, which we will discuss in detail in the ablation studies (see Section IV-E).

3) Results on Data-Assisted Progressive Inference Mechanism: The data-assisted mechanism aims to achieve more accurate predictions by utilizing a well-trained GCN classifier to aggregate features from both test data and labeled source data. We set the confidence threshold $\tau_p = 0.7$ and record the *Progressive Ratio* of data needing to be transmitted to the cloud. To evaluate the cloud-side classifiers, we calculate the average Acc^{PCG} and Acc^{PCM} based on the data transmitted from the terminal, as shown in Table VII. It is apparent that the *Progressive Ratio* consistently remains remarkably low, nearly always below 4%, indicating that the terminal's classifier can provide reliable predictions. In addition, the GCN classifier outperforms the MLP classifier, highlighting the advantage of the data-assisted progressive inference mechanism, where images from the source domain are sampled to assist predictions for the transmitted data.

By using the data-assisted progressive inference mechanism, we integrate the terminal-side classifier and the cloud-side

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
RESULTS OF DATA-ASSISTED PROGRESSIVE INFERENCE

	A → N, P, R, U	N → A, P, R, U	P → A, N, R, U
Progressive Ratio	2.03	1.24	3.99
Acc^{PCG}	60.27	68.27	49.82
Acc^{PCM}	55.01	54.07	35.12
	R → A, N, P, U	U → A, N, P, R	Average
Progressive Ratio	3.45	3.77	2.90
Acc^{PCG}	63.12	54.43	59.18
Acc^{PCM}	42.73	47.99	46.98

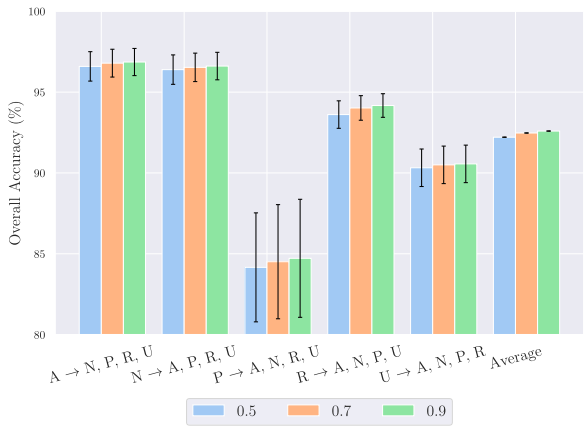


Fig. 7. Overall accuracy of TeCCo under varying thresholds τ_p.

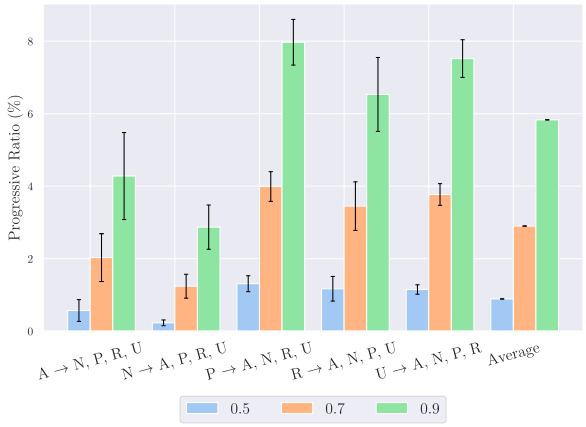


Fig. 8. Progressive ratio of TeCCo under varying thresholds τ_p.

TABLE VIII
COMPUTATIONAL COST OF DIFFERENT METHODS

Methods	Parameters	Flops	Training Time	Inference Time
Baseline	24.03M	4.10G	0.17s	0.10s
BSP	24.04M	4.11G	0.54s	0.17s
MDD	27.73M	4.11G	0.40s	0.12s
DAMC	30.67M	4.11G	0.46s	0.15s
AMRAN	26.82M	4.14G	0.45s	0.14s
SSOUDA	26.08M	4.11G	0.93s	0.15s
TeCCo (Cloud)	36.84M	5.93G	0.66s	-
TeCCo (Terminal)	11.31M	1.82G	-	0.03s

GCN classifier to calculate the overall accuracy of TeCCo. Given a dataset of size n, the terminal confidently predicts t images and transfers the remaining $c = n - t$ images to the cloud. Let t' be the number of correct predictions made by the terminal and c' the correct predictions made by the cloud. The overall accuracy is then $((t' + c')/n)\%$. Due to the correct predictions c provided by the cloud, TeCCo can achieve higher performance. To evaluate this improvement, we assess TeCCo's overall accuracy under different confidence thresholds τ_p as depicted in Fig. 7. It indicates that TeCCo achieves its best performance at $\tau_p = 0.9$. Additionally, the Progressive Ratio varies with different confidence thresholds τ_p as shown in Fig. 8. We note that a higher confidence threshold τ_p results in a higher Progressive Ratio, which necessitates the transmission of more data. To balance efficiency and performance, we use $\tau_p = 0.7$ as the default setting.

4) Analysis on Computational Cost: The computational cost is a crucial criterion for assessing practicality. We evaluate the computational cost using various metrics, including floating-point operations per second (FLOPs), the number of parameters, and both training and inference time. For a fair comparison, all experiments are conducted on the same P40 GPU. To measure FLOPs, we use the widely adopted tool.[1] We evaluate the training and inference time over every batch and set a batch size of 32 for all methods. Due to the distributed cloud-terminal framework of TeCCo, we can leverage the rich resources on the cloud for training and only use the terminal for fast inference.

As observed from Table VIII, during the training process on the cloud, TeCCo exhibits a slightly higher number of parameters due to its additional lightweight feature network, leading to increased FLOPs. However, it is important to note that TeCCo does not require significantly more training time than other methods, as almost all computational tasks can be handled by the GPU. Notably, during the inference process on the terminal, TeCCo achieves the shortest inference time, due to the use of its lightweight feature network during inference.

From this computational analysis, it is evident that TeCCo offers the most efficient inference time while maintaining a training cost comparable to other methods. Furthermore, TeCCo achieves higher accuracy compared to its counterparts as shown in Section IV-B. These findings underscore the practicality and effectiveness of TeCCo.

E. Ablation Study

In this section, we conduct ablation studies to investigate the impact of key components in TeCCo. First, we highlight improvements from integrating the DCCL module. Next, we explore the advantages of using the HNP module for individual terminals. Following this, we analyze the strategies employed in the training design for the personalized fine-tuning stage. Finally, we examine critical hyperparameters in TeCCo and explain our selection process. Notably, we use Acc^{T} as the evaluation metric in this section unless otherwise specified.

[1]https://github.com/facebookresearch/fvcore

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

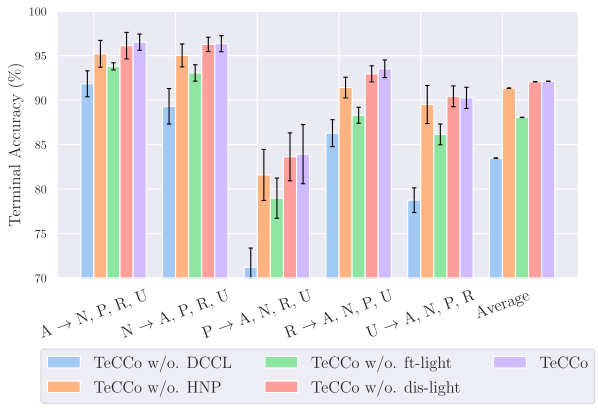


Fig. 9. Results of various TeCCo variants in ablation studies.

1) Dual Classifier Co-Learning: In the DCCL module, we employ EdgeNet to determine pairwise similarity between samples, which is then utilized by the NodeNet to predict the class of each sample. Compared to the MLP, the GCN produces more robust features by combining information from multiple samples. To assess the effect of DCCL on our techniques, we contrive a variant termed **TeCCo w/o. DCCL**. In this modified approach, we remove the GCN classifier and rely solely on the MLP classifier with the discriminator for training purposes. It is worth noting that this variation diverges from the Baseline displayed in Table III, as it merely omits the GCN but retains all other constituent modules of our methods, such as the hypernetwork and discriminator. In reviewing Fig. 9, we observe that the performance of **TeCCo w/o. DCCL** significantly declines compared to the original TeCCo. This aligns with the results in Fig. 6, where the GCN classifier clearly outperforms the MLP classifier in the cloud device. Hence, the GCN classifier can generate more accurate pseudo labels for the fine-tuning phase.

2) Hypernetwork Personalization: In Table VI, we verify the training efficiency of the HNP module. To further examine its performance, we devised a variant called **TeCCo w/o. HNP** which excludes the hypernetwork and instead employs a unique MLP classifier with a single linear layer for each terminal. The hypernetwork includes an embedding layer and a linear layer, requiring fine-tuning only the embedding layer's parameters for each terminal's classifier. In addition, utilizing a shared linear layer across terminals can enhance overall performance as shown in Fig. 9. It is evident that **TeCCo w/o. HNP** exhibits decreased performance across all settings, clearly highlighting the benefits of incorporating a hypernetwork in TeCCo.

3) Training Strategy for Lightweight Feature Network: In practical applications, terminals typically possess limited computational resources. To address this issue, we incorporate a lightweight feature network for terminals. TeCCo includes two training strategies for the lightweight feature network. We train it using distillation loss throughout the entire learning stage, and then add classification loss during the personalized fine-tuning stage. This approach ensures that the lightweight feature network can extract features similar to those of the larger feature network in the cloud and also learn unique information specific to each terminal, thereby enhancing performance for the terminal-side classifier. In this experiment, we rigorously investigate the impact of these two distinct training strategies.

TABLE IX
COMPARISON RESULTS ON THE PROGRESSIVE INFERENCE MECHANISM

Methods	A → N, P, R, U	N → A, P, R, U	P → A, N, R, U
TeCCo	60.27	68.27	49.82
TeCCo w/o. dis-light	25.28	35.14	25.51
Methods	**R → A, N, P, U**	**U → A, N, P, R**	**Average**
TeCCo	63.12	54.43	59.18
TeCCo w/o. dis-light	34.66	29.42	30.00

We begin by examining the necessity of using classification loss to refine the lightweight feature network during the personalized fine-tuning stage. This investigation leads to the creation of a variant named **TeCCo w/o. ft-light** in which we only optimize the parameters of the lightweight feature network using distillation loss. As shown in Fig. 9, **TeCCo w/o. ft-light** demonstrates a pronounced degradation in performance compared to the original TeCCo. This finding suggests the necessity of involving classification loss.

We then study the need for distillation loss and create the variant **TeCCo w/o. dis-light**. In this variant, the lightweight feature network is only optimized from scratch during the personalized stage. As observed in Fig. 9, **TeCCo w/o. dis-light** does not degrade the performance of the terminal-side classifier. However, omitting distillation loss significantly hampers the data-assisted progressive inference mechanism. We calculate Acc^{PCG} based on data transmitted from the terminal, as shown in Table IX. The **TeCCo w/o. dis-light** fails to achieve high accuracy when utilizing the cloud-side GCN classifier for more accurate inference. This indicates that, without distillation loss, TeCCo can still learn meaningful features for each terminal, but these features differ too much from those extracted by the cloud-side feature network, preventing the cloud-side GCN classifier from making accurate predictions based on terminal-transmitted features. We will further justify this in Section IV-F via visualization experiments.

In conclusion, both the **TeCCo w/o. dis-light** and **TeCCo w/o. ft-light** variants show performance degradation compared to the original TeCCo. This underscores the inadequacy of relying solely on classification or distillation loss for optimizing the lightweight feature network.

4) Hyperparameters: We begin by discussing the hyperparameters in the objective (19) of TeCCo. Since the loss terms $\mathcal{L}_{\text{mlp}}$ and $\mathcal{L}_{\text{adv}}$ are commonly used in the field of cross-domain learning [55], [92], [93], we set both λ_{mlp} and λ_{adv} to 1.0 as the default settings. Therefore, we focus on the impact of the three remaining weight coefficients. We first study the impact of λ_{edge} and λ_{node}, which are crucial for the DCCL module. As shown in Table X, the results indicate that all settings yield similar performance, with $\lambda_{\text{edge}} = 0.3$ and $\lambda_{\text{node}} = 0.3$ providing the best performance. We then select NWPU-RESISC45 as the source domain and evaluate TeCCo under different λ_{distill}. From Table XI, we observe that TeCCo still achieves similar performance under different λ_{distill} values, demonstrating its robustness.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

TABLE X
ABLATION STUDY ON λ_{node} AND λ_{edge} IN (19)

λ_{node} - λ_{edge}	A → N, P, R, U	N → A, P, R, U	P → A, N, R, U
0.1 - 0.1	93.47	93.80	82.44
0.3 - 0.3	96.52	96.35	83.93
0.5 - 0.5	95.17	96.01	83.78
0.7 - 0.7	96.23	95.63	83.63

λ_{node} - λ_{edge}	R → A, N, P, U	U → A, N, P, R	Average
0.1 - 0.1	90.77	88.52	89.80
0.3 - 0.3	93.53	90.26	92.12
0.5 - 0.5	95.11	88.31	91.67
0.7 - 0.7	91.24	84.04	90.16

TABLE XI
ABLATION STUDY ON λ_{distill}

λ_{distill}	AID	PatternNet	RSD46-WHU	UCMerced	Average
0.3	99.30	94.03	91.89	98.94	96.04
0.5	99.32	93.99	91.80	98.94	96.01
1.0	99.55	95.27	91.03	99.42	96.32

We observe that the initialization of the embedding layer in the hypernetwork is crucial, as it impacts the learning of unique domain-specific information. We evaluate various initial distributions for the embedding layer, including spherical, uniform, and Gaussian distributions. Moreover, we evaluate different sizes for the hypernetwork's embedding layer, selecting from the set $\{64, 128, 256\}$. The findings in Fig. 10(a) indicate that the initialization of the embedding layer influences performance to a certain degree, with the spherical distribution proving to be the best choice. Further analysis in Fig. 10(b) underscores the need to balance performance and efficiency. Therefore, we set 128 to embedding size as the default setting.

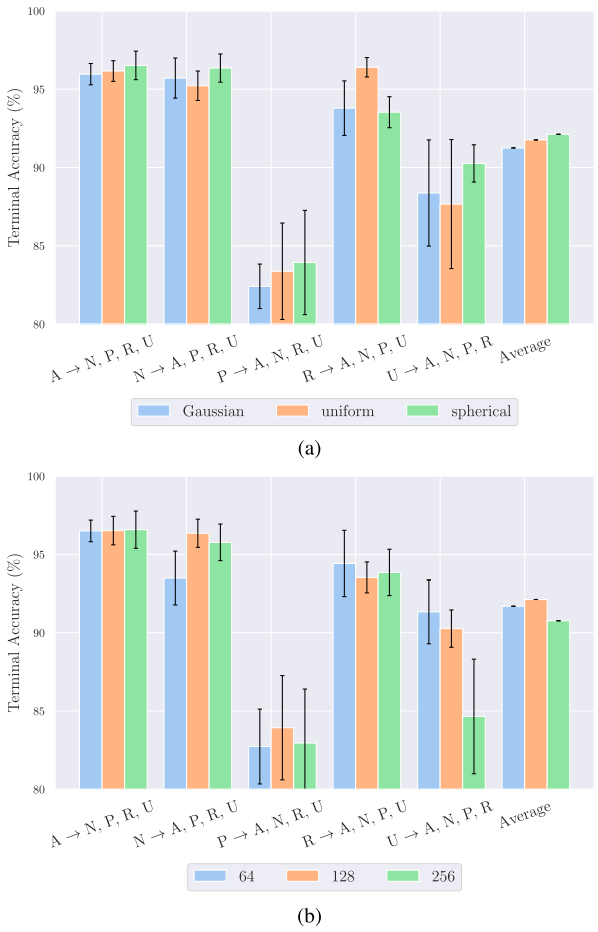


Fig. 10. Ablation study on the embedding layer of hypernetwork. (a) Experiment on the initialization method. (b) Experiment on the embedding size.

F. Visualization Analysis

In this section, we present visualization experiments aimed at gaining a deeper understanding of why TeCCo can achieve the superior performance.

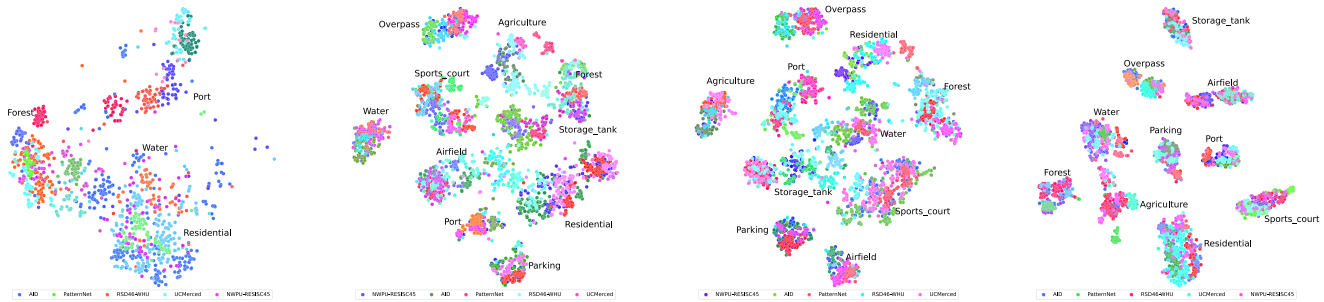
1) Generalization Across Different Domains: We have demonstrated significant discrepancies among images from the same categories but different domains in Fig. 4. Therefore, a pretrained model from one dataset cannot be directly applied to another dataset. To address this issue, TeCCo utilizes the DCCL module, which enables the model to efficiently learn common knowledge from different domains, thereby ensuring robustness across various domains. To better demonstrate the effectiveness of TeCCo in processing images from different domains, we employ t-SNE to visualize the feature space learned by different methods across different domains. In this experiment, we use NWPU-RESISC45 as the source domain and compare TeCCo against AMRAN [88] and SSOUDA [89]. We also visualize the features extracted by the Baseline, which utilizes only ResNet-50 as the feature network along with a single MLP classifier.

As depicted in Fig. 11, both AMRAN and SSOUDA demonstrate improved feature distributions across different domains compared to the Baseline. However, they still exhibit difficulties in distinguishing certain classes. For instance, AMRAN struggles to classify airfield, storage_tank, and residential categories within the feature space. SSOUDA is less effective at classifying water, sports_court, and storage_tank. This phenomenon is consistent with the findings presented in the confusion matrix in Fig. 5. The visualization results in Fig. 11(d) show that TeCCo can learn a similar feature space for images from the same category but different domains. These visual comparisons further highlight the advantages of TeCCo in RS image classification across different domains.

2) Feature Visualization From Cloud and Terminal: The large feature network on the cloud can learn common knowledge across different domains. It is crucial to transfer this capability to the lightweight feature network on the terminal. To achieve this, we employ distillation loss to optimize the lightweight feature network. As shown in Fig. 12(b), the lightweight feature network learns a feature space nearly identical to that of the large feature network.

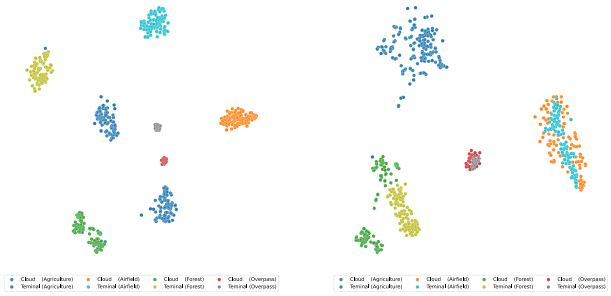
In addition, we compare the feature discrepancies between cloud and terminal in the variant **TeCCo w/o. dis-light**, where the distillation loss is not applied. As shown in Fig. 12(a), the lightweight feature network in **TeCCo w/o. dis-light** can still distinguish images from different domains. However, without the distillation loss, there is a significant

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c) (d)

Fig. 11. t-SNE visualization on features from different domains. (a) Baseline. (b) AMRAN. (c) SSOUDA. (d) TeCCo.



(a) (b)

Fig. 12. t-SNE visualization on features from cloud and terminal. (a) TeCCo w/o. dis-light. (b) TeCCo.

feature discrepancy between the cloud and the terminal. This phenomenon explains why **TeCCo w/o. dis-light** performs poorly in the data-assisted progressive inference mechanism, as demonstrated in Table IX. Therefore, the distillation loss is crucial for TeCCo to enhance the data-assisted progressive inference mechanism and achieve improved performance.

V. Conclusion

In this article, we proposed a terminal-cloud cross-domain collaborative learning framework named TeCCo to address the challenges of multiplatform RS image classification, particularly focusing on the data distribution differences across various RS devices. TeCCo enhances both the robustness of the cloud model and the personalization of terminal models, enabling efficient and accurate collaborative interpretation. Essential components of the TeCCo include the DCCL module, which enhances feature alignment and robustness, and the HNP module which generates customized classifier parameters for each terminal. In addition, a data-assisted progressive inference mechanism ensures higher accuracy on the cloud by jointly clustering features from terminals and supervised domain data. For future work, we plan to explore the application of multiplatform collaborative interpretation frameworks to other RS tasks such as semantic segmentation, object detection, and change detection, further enhancing the versatility and effectiveness of our approach.

References

[1] H. Tan and S. Li, "Interferometric calibration of wide swath altimeters with inclined baseline using inland lakes," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2023, pp. 4290–4293.

[2] H. Wang, S. Liu, Y. Lv, and S. Li, "Scattering information fusion network for oriented ship detection in SAR images," *IEEE Geosci. Remote Sens. Lett.*, vol. 20, pp. 1–5, 2023.

[3] Z. Zou and Z. Shi, "Ship detection in spaceborne optical image with SVD networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 5832–5845, Oct. 2016.

[4] H. Lin, Z. Shi, and Z. Zou, "Fully convolutional network with task partitioning for inshore ship detection in optical remote sensing images," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 10, pp. 1665–1669, Oct. 2017.

[5] Y. Gu, Q. Wang, X. Jia, and J. A. Benediktsson, "A novel MKL model of integrating LiDAR data and MSI for urban area classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 10, pp. 5312–5326, Oct. 2015.

[6] Y. Gu and Q. Wang, "Discriminative graph-based fusion of HSI and LiDAR data for urban area classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 6, pp. 906–910, Jun. 2017.

[7] J. Kang, Z. Wang, R. Zhu, X. Sun, R. Fernandez-Beltran, and A. Plaza, "PiCoCo: Pixelwise contrast and consistency learning for semisupervised building footprint segmentation," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 10548–10559, 2021.

[8] Y. Kang, Z. Wang, J. Fu, X. Sun, and K. Fu, "SFR-Net: Scattering feature relation network for aircraft detection in complex SAR images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5218317.

[9] Y. Sun, Z. Wang, X. Sun, and K. Fu, "SPAN: Strong scattering point aware network for ship detection and classification in large-scale SAR imagery," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 15, pp. 1188–1204, 2022.

[10] T. Zhang, Z. Wang, P. Cheng, G. Xu, and X. Sun, "DCNNet: A distributed convolutional neural network for remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5603618.

[11] Y. Wang et al., "DCM: A distributed collaborative training method for the remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5605018.

[12] Z. Niu et al., "FCIL-MSN: A federated class-incremental learning method for multisatellite networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5626115.

[13] X. Sun et al., "RingMo: A remote sensing foundation model with masked image modeling," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2022, Art. no. 5612822.

[14] A. Ghodrati, B. E. Bejnordi, and A. Habibian, "FrameExit: Conditional early exiting for efficient video recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2021, pp. 15608–15618.

[15] S. Zhang et al., "Towards real-time cooperative deep inference over the cloud and edge end devices," *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 4, no. 2, pp. 1–24, Jun. 2020.

[16] A. E. Eshratifar, M. S. Abrishami, and M. Pedram, "JointDNN: An efficient training and inference engine for intelligent mobile cloud computing services," *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 565–576, Feb. 2021.

[17] L. Zhang, Z. Tan, J. Song, J. Chen, C. Bao, and K. Ma, "SCAN: A scalable neural networks framework towards compact and efficient models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, Jan. 2019, pp. 11–12.

[18] A. Dehkordi, N. Vedula, J. Pei, F. Xia, L. Wang, and Y. Zhang, "Auto-Split: A general framework of collaborative edge-cloud AI," in *Proc. ACM SIGKDD Conf. KDD*, 2021, pp. 2543–2553.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

[19] F. Zhang et al., "Multitarget domain adaptation building instance extraction of remote sensing imagery with domain-common approximation learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 4702916.

[20] D. Biswas and J. Tešić, "Unsupervised domain adaptation with debiased contrastive learning and support-set guided pseudolabeling for remote sensing images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 17, pp. 3197–3210, 2024.

[21] J. Zheng et al., "A two-stage adaptation network (TSAN) for remote sensing scene classification in single-source-mixed-multiple-target domain adaptation (S^2M^2T DA) scenarios," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5609213.

[22] J. Zhang, J. Liu, B. Pan, and Z. Shi, "Domain adaptation based on correlation subspace dynamic distribution alignment for remote sensing image scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 11, pp. 7920–7930, Nov. 2020.

[23] Q. Xu, Y. Shi, X. Yuan, and X. X. Zhu, "Universal domain adaptation for remote sensing image scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 4700515.

[24] W. Huang, Y. Shi, Z. Xiong, Q. Wang, and X. X. Zhu, "Semi-supervised bidirectional alignment for remote sensing cross-domain scene classification," *ISPRS J. Photogramm. Remote Sens.*, vol. 195, pp. 192–203, Jan. 2023.

[25] J. Zhang et al., "FedCP: Separating feature information for personalized federated learning via conditional policy," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2023, pp. 3249–3261.

[26] J. Yao et al., "Edge-cloud polarization and collaboration: A comprehensive survey for AI," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 7, pp. 6866–6886, Aug. 2022.

[27] B. Zhang, T. Xiang, H. Zhang, T. Li, S. Zhu, and J. Gu, "Dynamic DNN decomposition for lossless synergistic inference," in *Proc. IEEE 41st Int. Conf. Distrib. Comput. Syst. Workshops (ICDCSW)*, Jun. 2021, pp. 13–20.

[28] W. Guo et al., "Remote sensing image scene classification by multiple granularity semantic learning," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 15, pp. 2546–2562, 2022.

[29] Y. Hu, Z. Li, Y. Chen, Y. Cheng, Z. Cao, and J. Liu, "Content-aware adaptive device-cloud collaborative inference for object detection," *IEEE Internet Things J.*, vol. 10, no. 21, pp. 19087–19101, Nov. 2023.

[30] Y. Kang et al., "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," *ACM SIGPLAN Notices*, vol. 52, no. 4, pp. 615–629, May 2017.

[31] G. Liu et al., "An adaptive DNN inference acceleration framework with end–edge–cloud collaborative computing," *Future Gener. Comput. Syst.*, vol. 140, pp. 422–435, Mar. 2023.

[32] Z. Zhao, K. Wang, N. Ling, and G. Xing, "EdgeML: An AutoML framework for real-time deep learning on the edge," in *Proc. Int. Conf. Internet-Things Design Implement.*, May 2021, pp. 133–144.

[33] R. Mehta and R. Shorey, "DeepSplit: Dynamic splitting of collaborative edge-cloud convolutional neural networks," in *Proc. Int. Conf. Commun. Syst. Netw. (COMSNETS)*, Jan. 2020, pp. 720–725.

[34] Y. Zhang, J.-H. Liu, C.-Y. Wang, and H.-Y. Wei, "Decomposable intelligence on cloud-edge IoT framework for live video analytics," *IEEE Internet Things J.*, vol. 7, no. 9, pp. 8860–8873, Sep. 2020.

[35] Z. Zhao, K. M. Barijough, and A. Gerstlauer, "DeepThings: Distributed adaptive deep learning inference on resource-constrained IoT edge clusters," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 11, pp. 2348–2359, Nov. 2018.

[36] Y. Huang et al., "Cross-scene wetland mapping on hyperspectral remote sensing images using adversarial domain adaptation network," *ISPRS J. Photogramm. Remote Sens.*, vol. 203, pp. 37–54, Sep. 2023.

[37] Y. Ning, J. Peng, L. Sun, Y. Huang, W. Sun, and Q. Du, "Adaptive local discriminant analysis and distribution matching for domain adaptation in hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 15, pp. 4797–4808, 2022.

[38] H. Wang, Y. Cheng, C. L. P. Chen, and X. Wang, "Hyperspectral image classification based on domain adversarial broad adaptation network," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5517813.

[39] Y. Huang et al., "Two-branch attention adversarial domain adaptation network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5540813.

[40] Y. Huang, J. Peng, Y. Ning, W. Sun, and Q. Du, "Graph embedding and distribution alignment for domain adaptation in hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 7654–7666, 2021.

[41] J. Peng, Y. Huang, W. Sun, N. Chen, Y. Ning, and Q. Du, "Domain adaptation in remote sensing image classification: A survey," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 15, pp. 9842–9859, 2022.

[42] Y. Cui, L. Wang, J. Su, S. Gao, and L. Wang, "Iterative weighted active transfer learning hyperspectral image classification," *J. Appl. Remote Sens.*, vol. 15, no. 3, Jul. 2021, Art. no. 032207.

[43] H. Li, J. Li, Y. Zhao, M. Gong, Y. Zhang, and T. Liu, "Cost-sensitive self-paced learning with adaptive regularization for classification of image time series," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 11713–11727, 2021.

[44] Z. Deng, K. Zhou, D. Li, J. He, Y.-Z. Song, and T. Xiang, "Dynamic instance domain adaptation," *IEEE Trans. Image Process.*, vol. 31, pp. 4585–4597, 2022.

[45] J. Wang, Y. Chen, H. Yu, M. Huang, and Q. Yang, "Easy transfer learning by exploiting intra-domain structures," in *Proc. IEEE Int. Conf. Multimedia Expo. (ICME)*, Jul. 2019, pp. 1210–1215.

[46] G. Matasci, D. Tuia, and M. Kanevski, "SVM-based boosting of active learning strategies for efficient domain adaptation," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 5, no. 5, pp. 1335–1343, Oct. 2012.

[47] B. Fernando, A. Habrard, M. Sebban, and T. Tuytelaars, "Unsupervised visual domain adaptation using subspace alignment," in *Proc. IEEE Int. Conf. Comput. Vis.*, Dec. 2013, pp. 2960–2967.

[48] M. Long, J. Wang, G. Ding, J. Sun, and P. S. Yu, "Transfer feature learning with joint distribution adaptation," in *Proc. IEEE Int. Conf. Comput. Vis.*, Dec. 2013, pp. 2200–2207.

[49] J. Zhang, W. Li, and P. Ogunbona, "Joint geometrical and statistical alignment for visual domain adaptation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1859–1867.

[50] J. Li, M. Jing, K. Lu, L. Zhu, and H. T. Shen, "Locality preserving joint transfer for domain adaptation," *IEEE Trans. Image Process.*, vol. 28, no. 12, pp. 6103–6115, Dec. 2019.

[51] M. Long, Y. Cao, Z. Cao, J. Wang, and M. I. Jordan, "Transferable representation learning with deep adaptation networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 12, pp. 3071–3085, Dec. 2019.

[52] Y. Zhu et al., "Deep subdomain adaptation network for image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 4, pp. 1713–1722, Apr. 2021.

[53] Y. Zhu et al., "Multi-representation adaptation network for cross-domain image classification," *Neural Netw.*, vol. 119, pp. 214–221, Nov. 2019.

[54] N. Xiao and L. Zhang, "Dynamic weighted learning for unsupervised domain adaptation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 15242–15251.

[55] E. Tzeng, J. Hoffman, K. Saenko, and T. Darrell, "Adversarial discriminative domain adaptation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 7167–7176.

[56] B. Liu, X. Yu, A. Yu, P. Zhang, G. Wan, and R. Wang, "Deep few-shot learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2290–2304, Apr. 2019.

[57] C. Yu, J. Wang, Y. Chen, and M. Huang, "Transfer learning with dynamic adversarial adaptation network," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2019, pp. 778–786.

[58] K. Saito, K. Watanabe, Y. Ushiku, and T. Harada, "Maximum classifier discrepancy for unsupervised domain adaptation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3723–3732.

[59] Y. Huang, J. Peng, G. Zhang, W. Sun, N. Chen, and Q. Du, "Adversarial domain adaptation network with calibrated prototype and dynamic instance convolution for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5514613.

[60] D. Ha, A. Dai, and Q. V. Le, "HyperNetworks," 2016, *arXiv:1609.09106*.

[61] Y. Alaluf, O. Tov, R. Mokady, R. Gal, and A. Bermano, "HyperStyle: StyleGAN inversion with HyperNetworks for real image editing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 18490–18500.

[62] J. von Oswald, C. Henning, B. F. Grewe, and J. Sacramento, "Continual learning with hypernetworks," 2019, *arXiv:1906.00695*.

[63] O. Chang, L. Flokas, and H. Lipson, "Principled weight initialization for hypernetworks," 2023, *arXiv:2312.08399*.

[64] A. Shamsian, A. Navon, E. Fetaya, and G. Chechik, "Personalized federated learning using hypernetworks," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 9489–9502.

[65] C. Zhang, M. Ren, and R. Urtasun, "Graph HyperNetworks for neural architecture search," 2018, *arXiv:1810.05749*.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.

[66] Z. Xian, S. Lal, H.-Y. Tung, E. Antonios Platanios, and K. Fragkiadaki, "HyperDynamics: Meta-learning object and agent dynamics with hypernetworks," 2021, *arXiv:2103.09439*.

[67] S. Laskaridis, S. I. Venieris, M. Almeida, I. Leontiadis, and N. D. Lane, "SPINN: Synergistic progressive inference of neural networks over device and cloud," in *Proc. 26th Annu. Int. Conf. Mobile Comput. Netw.*, Sep. 2020, pp. 1–15.

[68] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "Distributed deep neural networks over the cloud, the edge and end devices," in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Apr. 2017, pp. 328–339.

[69] Y. Ganin et al., "Domain-adversarial training of neural networks," *J. Mach. Learn. Res.*, vol. 17, no. 59, pp. 1–35, 2016.

[70] G.-S. Xia et al., "AID: A benchmark data set for performance evaluation of aerial scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3965–3981, Jul. 2017.

[71] G. Cheng, J. Han, and X. Lu, "Remote sensing image scene classification: Benchmark and state of the art," *Proc. IEEE*, vol. 105, no. 10, pp. 1865–1883, Oct. 2017.

[72] W. Zhou, S. Newsam, C. Li, and Z. Shao, "PatternNet: A benchmark dataset for performance evaluation of remote sensing image retrieval," *ISPRS J. Photogramm. Remote Sens.*, vol. 145, pp. 197–209, Nov. 2018.

[73] Y. Long, Y. Gong, Z. Xiao, and Q. Liu, "Accurate object localization in remote sensing images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 5, pp. 2486–2498, May 2017.

[74] Z. Xiao, Y. Long, D. Li, C. Wei, G. Tang, and J. Liu, "High-resolution remote sensing image retrieval based on CNNs from a dimensional perspective," *Remote Sens.*, vol. 9, no. 7, p. 725, Jul. 2017.

[75] Y. Yang and S. Newsam, "Bag-of-visual-words and spatial extensions for land-use classification," in *Proc. 18th SIGSPATIAL Int. Conf. Adv. Geograph. Inf. Syst.*, Nov. 2010, pp. 270–279.

[76] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 86, pp. 2579–2605, Jan. 2008.

[77] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[78] E. Tzeng, J. Hoffman, N. Zhang, K. Saenko, and T. Darrell, "Deep domain confusion: Maximizing for domain invariance," 2014, *arXiv:1412.3474*.

[79] M. Long, Y. Cao, J. Wang, and M. Jordan, "Learning transferable features with deep adaptation networks," in *Proc. 32nd Int. Conf. Mach. Learn.*, vol. 37, Jul. 2015, pp. 97–105.

[80] B. Sun and K. Saenko, "Deep CORAL: Correlation alignment for deep domain adaptation," in *Proc. Comput. Vis. Workshops (ECCV)*, Amsterdam, The Netherlands. Cham, Switzerland: Springer, 2016, pp. 443–450.

[81] M. Long, H. Zhu, J. Wang, and M. I. Jordan, "Deep transfer learning with joint adaptation networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 2208–2217.

[82] M. Long, Z. Cao, J. Wang, and M. I. Jordan, "Conditional adversarial domain adaptation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 1640–1650.

[83] X. Wang, L. Li, W. Ye, M. Long, and J. Wang, "Transferable attention for domain adaptation," in *Proc. AAAI Conf. Artif. Intell.*, Jul. 2019, vol. 33, no. 1, pp. 5345–5352.

[84] K. Saito, S. Yamamoto, Y. Ushiku, and T. Harada, "Open set domain adaptation by backpropagation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 153–168.

[85] T. Shermin, G. Lu, S. W. Teng, M. Murshed, and F. Sohel, "Adversarial network with multiple classifiers for open set domain adaptation," *IEEE Trans. Multimedia*, vol. 23, pp. 2732–2744, 2021.

[86] X. Chen, S. Wang, M. Long, and J. Wang, "Transferability vs. discriminability: Batch spectral penalization for adversarial domain adaptation," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 1081–1090.

[87] Y. Zhang, T. Liu, M. Long, and M. Jordan, "Bridging theory and algorithm for domain adaptation," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 7404–7413.

[88] S. Zhu, B. Du, L. Zhang, and X. Li, "Attention-based multiscale residual adaptation network for cross-scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5400715.

[89] S. Wang, D. Hou, and H. Xing, "A self-supervised-driven open-set unsupervised domain adaptation method for optical remote sensing image scene classification and retrieval," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5605515.

[90] K. Liu, J. Yang, and S. Li, "Remote-sensing cross-domain scene classification: A dataset and benchmark," *Remote Sens.*, vol. 14, no. 18, p. 4635, Sep. 2022.

[91] F. Wang, Y. Liang, Y. Cui, S. Li, and R. Wang, "Unsupervised domain adaptation remote sensing image classification based on class centroid alignment," in *Proc. China Autom. Congr. (CAC)*, Nov. 2023, pp. 1898–1903.

[92] L. Zhou, M. Ye, X. Zhu, S. Li, and Y. Liu, "Class discriminative adversarial learning for unsupervised domain adaptation," in *Proc. 30th ACM Int. Conf. Multimedia*, Oct. 2022, pp. 4318–4326.

[93] L. Shi and W. Liu, "Adversarial self-training improves robustness and generalization for gradual domain adaptation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–17.



Ting Zhang received the B.Eng. degree from Tianjin University, Tianjin, China, in 2020. She is currently pursuing the joint Ph.D. degree with the University of Chinese Academy of Sciences, Beijing, China, and the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing.

Her research interests include computer vision and pattern recognition, especially on remote sensing image interpretation.



Peirui Cheng received the B.S. degree from the School of Information Science and Technology, University of Science and Technology of China, Hefei, China, in 2013, the M.S. degree from the University of Chinese Academy of Sciences, Beijing, China, in 2016, and the Ph.D. degree from the School of Computer Science and Technology, University of Chinese Academy of Sciences, in 2022.

He is currently a Post-Doctoral Researcher with the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing. His research interests mainly include scene text detection and remote sensing.



Liangjin Zhao received the B.E. degree in automation from the University of Electronic Science and Technology of China, Chengdu, China, in 2015, and the master's degree from Beijing Institute of Technology, Beijing, China, in 2018.

He is currently a Research Assistant with the Institute of Electronics, Chinese Academy of Sciences, Beijing. His research interests include target detection and recognition in unmanned aerial vehicle remote sensing images and simultaneous localization and mapping.



Zhirui Wang (Member, IEEE) received the B.Sc. degree from Harbin Institute of Technology, Harbin, China, in 2013, and the Ph.D. degree from Tsinghua University, Beijing, China, in 2018.

He is currently an Associate Researcher with the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing. His research interests focus on the intelligent interpretation of multimodal remote sensing data.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.



Lingyu Kong (Student Member, IEEE) received the B.Sc. degree from Dalian Maritime University, Dalian, China, in 2020. She is currently pursuing the joint Ph.D. degree with the University of Chinese Academy of Sciences, Beijing, China, and the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing.

Her research interests include computer vision and pattern recognition, especially on remote sensing object detection and multiobject tracking.



Chenglong Liu (Graduate Student Member, IEEE) received the B.Eng. degree from Xidian University, Xi'an, China, in 2020. He is currently pursuing the joint Ph.D. degree with the University of Chinese Academy of Sciences, Beijing, China, and the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing.

His research interests include computer vision, pattern recognition, remote sensing image processing, multimodal learning, and large vision-language models.



Guangluan Xu received the B.Sc. degree from Beijing Information Science and Technology University, Beijing, China, in 2000, and the M.Sc. and Ph.D. degrees (five-year Ph.D. Program) from the Institute of Electronics, Chinese Academy of Sciences, Beijing, in 2005.

He is currently a Professor with the Aerospace Information Research Institute, Chinese Academy of Sciences. His research interests include computer vision and remote sensing image understanding.



Xian Sun (Senior Member, IEEE) received the B.Sc. degree from Beijing University of Aeronautics and Astronautics, Beijing, China, in 2004, and the M.Sc. and Ph.D. degrees from the Institute of Electronics, Chinese Academy of Sciences (CAS), Beijing, in 2009.

He was a Visiting Scholar with Karlsruher Institutfür Technologie, Karlsruhe, Germany, in 2013. He is currently a Professor with the Aerospace Information Research Institute, CAS. His research interests include computer vision, geospatial data mining, and remote sensing image understanding.

Dr. Sun was a recipient of the Outstanding Science and Technology Achievement Prize of the CAS in 2016 and the First Prize for The State Scientific and Technological Progress of China in 2019. He serves as an Associate Editor for IEEE ACCESS and a Guest Editor for the Special Issue of IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING (JSTARS) and other journals.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 10:59:50 UTC from IEEE Xplore. Restrictions apply.